ChatGPT

AI Triage System Configuration for NHS GP Deployment

1. Parameter Catalog by Module

Below is a comprehensive list of configuration parameters grouped by system module. Each parameter includes its meaning, units (if applicable), a safe default value with recommended range, any dependencies, and whether **clinical safety sign-off** is required for changes.

SafeguardGate (Urgent Safety Gate)

- **RED_FLAG_SET** (list of strings) – Lexicon of high-risk **"red flag"** symptoms or phrases that trigger immediate diversion to emergency care [1]. Used by the NLP to flag emergency keywords (e.g. "chest pain", "severe bleeding"). **Safe default:** a vetted list of emergency symptoms (e.g. chest pain, shortness of breath) [2]. **Range:** N/A (qualitative list, to be maintained by clinicians). **Dependencies:** Language support (should cover terms in all patient languages if `MULTILINGUAL_MODE="native"`). **Clinical sign-off:** Yes – any updates require clinical review to avoid missing critical symptoms.

- **RED_FLAG_THRESHOLD** (dimensionless, 0–1) – Confidence threshold for the NLP entity extractor to flag a **red-flag symptom** in text [3]. If the model's confidence that a red-flag symptom is present exceeds this value, the patient is diverted. **Safe default:** `0.65` [4] (moderate sensitivity). **Recommended range:** 0.5–0.8 (lower for more sensitivity, higher to reduce false alarms). **Dependencies:** Tied to NER model calibration (Platt scaling) [3]. **Clinical sign-off:** Yes – affects emergency diversion sensitivity.

- **MODEL_NER** (string) – Identifier for the clinical NLP model used for **symptom/entity extraction**. **Meaning:** which transformer model backbone to use for named entity recognition (e.g. "bioclinicalbert") [5]. **Default:** `"bioclinicalbert"` (UK-safe biomedical BERT). **Range:** any model name pre-integrated (must be clinically validated if changed). **Dependencies:** Must align with `MULTILINGUAL_MODE`. **Clinical sign-off:** *Conditional* – switching to a different model requires evidence of equal or better clinical performance.

- **MODEL_CLASSIFIER** (string) – Identifier for the **emergency text classifier** model (detects overall emergency context) [5]. **Default:** `"medalpaca-sm"` (lightweight transformer) [5]. **Range:** model must be approved for medical emergency detection. **Dependencies:** Language support (if not multilingual, see `TRANSLATE_ENGINE`). **Clinical sign-off:** *Conditional* – new model needs validation (sensitivity/specificity for emergencies).

- **MULTILINGUAL_MODE** (enum: `native|translate`) – Strategy for handling non-English submissions [6] [7]. `native`: use a multilingual NER model (e.g. XLM-R); `translate`: translate input to English then run monolingual model. **Default:** `"native"` (prefer multilingual model directly) [8]. **Dependencies:** If set to `translate`, must configure `TRANSLATE_ENGINE`. **Clinical sign-off:** No (does not alter thresholds, but translation quality must be assessed).

- **TRANSLATE_ENGINE** (string) – Translation service to use if `MULTILINGUAL_MODE=translate` [6]. **Default:** `"none"` (not used unless mode is translate). **Options:** e.g. `"NHS_Lingua"` or third-party engines. **Dependencies:** Ensure medical terms are preserved; run lexicon checks post-translation [7]. **Clinical sign-off:** No (but require IG approval if cloud service).

- **EMERGENCY_CONFIDENCE** (dimensionless, 0–1) – Confidence threshold for the whole-text **emergency classifier** to decide "emergency likely" [3]. If `Classifier(doc) >= 0.70`, treat as emergency. **Safe default:** `0.70` [9]. **Range:** 0.6–0.9 (tune to achieve near-zero false negatives). **Dependencies:** Must be calibrated with RED_FLAG_SET (the classifier provides redundancy) [10] [3]. **Clinical sign-off:** Yes – ensures urgent cases aren't missed.

- **ACUITY_MODEL** (string) – The model or algorithm used for **acuity prediction** (severity assessment) [11]. **Default:** `"xgboost"` (gradient-boosted ensemble) [11] trained on historical triage outcomes. **Range:** any model name integrated (requires retraining with local data if changed). **Clinical sign-off:** *Conditional* – new model requires performance evaluation.

- **ACUITY_THRESHOLD_EMERGENCY** (dimensionless, 0–1) – Threshold on the acuity model's output to classify a case as **"Emergency" acuity** [11] [3]. If predicted acuity ≥ 0.75, treat as emergency-level. **Default:** `0.75` [11] (tuned for high specificity). **Range:** ~0.7–0.9 depending on model calibration (choose lower if model tends to under-call emergencies). **Dependencies:** Determination of `severity_prediction == "Emergency"` in SafeguardGate logic [12]. **Clinical sign-off:** Yes – defines cutoff for life-threatening risk.

- **TIMEOUT_MS** (integer, ms) – **Time budget** for SafeguardGate processing [13] [14]. If NLP + models exceed this time (e.g. 800 ms), the gate will **fail-safe**. **Default:** `800` ms [13] (p95 latency budget). **Range:** 500–1000 ms (keep low to avoid delaying urgent care) [14]. **Dependencies:** Hardware capacity (set based on model runtime). **Clinical sign-off:** Yes – if increased too high, could slow urgent response; if too low, might skip safety checks.

- **FALLBACK** (enum: `rules|none`) – Behavior if SafeguardGate fails or times out [15] [14]. `rules`: apply a simpler rules-based check (using `RED_FLAG_SET` lexicon and basic logic) if the ML gate fails; `none`: just proceed without gating if timed out. **Default:** `"rules"` [15] (ensures **safety override** even on failure). **Clinical sign-off:** Yes – turning off fallback is high risk (would allow potential emergencies through on an ML failure).

Rationale & Confidence: The above SafeguardGate defaults prioritize **patient safety** (erring on sensitivity). For example, **0.65/0.70** thresholds for red flags and emergency classifier are derived from calibration to minimize missed emergencies [3]. These values align with NHS England's requirement to **block urgent queries from digital submissions** [16] – ensuring any hint of an emergency (e.g. "severe chest pain") triggers immediate diversion with 999 or A&E advice [1]. **Confidence:** High (based on clinical safety standards and calibration; will be tuned further with real data). All safety-related parameters are marked for clinical safety officer sign-off under DCB0129 governance.

Unified Triage Scoring & SLA Aging

- **PRIORITY_THRESHOLDS** (dimensionless scores) – Cut-off values mapping the composite **triage score** to priority labels [17]. Defined as a map `{STAT, URGENT, SOON, ROUTINE}` where each value is the minimum score needed for that priority. **Safe default:** `{stat: 0.9, urgent: 0.7, soon: 0.4, routine: 0.0}` [18]. This means score ≥0.9 → **STAT**, ≥0.7 → **URGENT**, ≥0.4 → **SOON**, else ROUTINE. **Recommended range:** Calibrate to achieve desired caseload distribution (e.g. ~5% stat, 15% urgent, etc., depending on practice data). **Dependencies:** Must align with **SLA_TARGETS** (e.g. a score just below 0.7 means patient will be treated as "soon" and handled within same day). **Clinical sign-off:** Yes – essentially defines clinical acuity bands.

- **SLA_TARGETS** (time durations) – **Service level targets** for first response by priority. Keys typically specify the priority and the expected response window. **Default (examples):** `urgent_first_contact: PT2H` (urgent seen within 2 hours), `routine_initial_response: P2D` (routine addressed within 2 days) [19] . **Recommended values:** Based on NHS England guidance and practice standards – e.g. STAT: immediate (0 hours), URGENT: 2 hours, SOON: same working day (~12 hours), ROUTINE: 48 hours [19] . These defaults align with the **"urgent within 2 hours" and "routine within 48 hours"** targets from NHS reports [19] and common GP triage practice (many GP clinics commit to respond to routine requests within 2 working days [20]). **Units:** ISO 8601 durations (e.g. `PT2H`, `P2D`). **Dependencies:** Priority labels from `PRIORITY_THRESHOLDS`. **Clinical sign-off:** Yes – tied to patient safety and contractual standards (should reflect at least minimum standard of care).

- **TRIAGE_SCORE_WEIGHTS** (dimensionless coefficients) – Weightings `w1...w5` for each component of the composite triage score formula [17] . Format: `{ acuity: w1, risk: w2, complexity: w3, time: w4, capacity: w5 }`. These control the influence of each factor:

- *w1 (acuity)* – weight of ML acuity score (clinical severity prediction).
- *w2 (risk)* – weight of binary/high-risk flags (e.g. red-flag terms).
- *w3 (complexity)* – weight of patient complexity (comorbidities, etc.).
- *w4 (time)* – weight of waiting time (SLA aging factor).

- *w5 (capacity)* – weight of capacity gap (negative impact). **Safe default:** for example, `{acuity: 1.0, risk: 0.5, complexity: 0.2, time: 0.5, capacity: 0.2}` (relative values). This default assumes acuity is the strongest determinant, with moderate boost from explicit risk flags and wait time, and smaller adjustments for complexity and capacity. **Recommended range:** weights should be non-negative and tuned so that a delay in response *monotonically* increases score, but not so high that trivial waits trump clinical urgency. Typically, `w1` ~ 1 (baseline), `w2` ~ 0.3–0.7, `w3` ~ 0.1–0.3, `w4` ~ 0.3–0.6, `w5` ~ 0.1–0.3. **Dependencies:** The scale of model outputs (e.g. acuityScore range) – weights may need normalization. **Clinical sign-off:** Yes – these determine how clinical risk factors translate to priority; must be reviewed for safe triage outcomes.

- **TimeDecay function** (function of time, monotonic) – Although not a static parameter, the **time-based score booster** is a configurable function. By default, define `TimeDecay(Δt)` as an increasing function that approaches 1.0 as wait time Δt reaches the maximum acceptable wait for the current priority. **Example:** a linear ramp: `TimeDecay(t) = min(1.0, t / T_max)` where `T_max` is a reference time (e.g. 48h for routine cases) or tied to SLA targets per priority. This ensures that as a request nears or exceeds its SLA (e.g. 48 hours for routine), the time factor contributes a full weight, effectively bumping the priority [21] [22] . **Safe default:** use SLA target of the current priority as `T_max` (e.g. for a routine task, 48h; for urgent, 2h) so that the score will cross into the next priority band as it breaches its deadline. **Monotonicity:** Guaranteed by design (non-decreasing in time). **Clinical sign-off:** Yes – implicit, as this function ensures **monotonic escalation** so no patient is overlooked if waiting too long [22] .

- **CapacityGap function** (function of demand & supply) – Also a functional config: computes a **demand–supply gap score** for required skills. For instance: `CapacityGap(skill, team) =`

`max(0, TasksWaiting(skill) / ProvidersAvailable(skill) - 1)`. A simple approach is to use the ratio of open tasks to available providers for that skill minus 1 (so 0 means supply meets or exceeds demand, and positive values indicate shortage). This gap is then scaled by weight `w5` and *subtracted* in the triage score [17] – meaning if there's a severe capacity shortfall, the score (hence priority) is slightly reduced, reflecting that the system may need to allocate resources differently. **Safe default:** a ratio-based gap as above, capped at some maximum (to avoid over-penalizing). **Monotonicity:** If demand increases or supply decreases, gap value rises (thus reducing score more). **Dependency:** Availability data from schedules. **Clinical sign-off:** Yes (indirectly) – this is more of an operational parameter, but its effect on prioritization should be reviewed so it doesn't unfairly delay high-risk cases (it should be a mild factor).

- **SIM_THRESHOLD** (dimensionless, 0–1) – Similarity threshold for **cross-channel deduplication** [23]. If a new request is ≥ this similar to a recent open request from the same patient, it will be merged instead of creating a new task. **Default:** `0.8` (e.g. 80% text similarity). **Range:** ~0.7–0.9 (depending on false-merge vs duplicate tolerance). **Dependencies:** Text similarity algorithm (could be embedding cosine similarity or token overlap). **Clinical sign-off:** Yes – merging two separate concerns by mistake could be a safety issue; threshold must be conservative and validated.

- **DEDUP_WINDOW** (time duration) – Look-back window for considering recent requests in deduplication [23]. Defines how far back to search for potential duplicates. **Default:** e.g. `PT72H` (72 hours). **Range:** a few days up to a week, depending on typical repeat contact patterns. **Dependencies:** none major (performance of lookup). **Clinical sign-off:** No (primarily affects workflow efficiency, not immediate safety).

- **AGING_INTERVAL** (time duration) – Frequency at which the system recomputes scores to apply **SLA aging** escalation [24]. **Default:** `PT5M` (every 5 minutes) [24]. **Range:** 1–15 minutes (shorter ensures prompt escalation; longer reduces CPU load). **Dependencies:** None (background job). **Clinical sign-off:** No (operational tuning; 5 minutes is already aggressive escalation frequency, as per safety requirement).

Rationale & Confidence: The triage parameters ensure a **unified scoring** that is fair and risk-aligned. The default priority thresholds and SLA targets are drawn from NHS best practices – e.g. urgent cases to be contacted within 2 hours and routine cases within 48 hours [19]. Real GP deployments use similar standards (e.g. patients are told they'll hear back within 48 hours for routine queries [20]). Confidence in these defaults is **high**, backed by NHS England's Enhanced Access specifications and common practice. The composite score weights are initially set based on expert judgment to prioritize clinical acuity and risk over other factors. Confidence in weights is **medium** – they will be refined with local data (initial shadow mode tuning). The monotonic time decay function guarantees no patient "falls through the cracks" – if a case waits too long, it will escalate automatically [22]. This aligns with safety requirements and thus confidence in the principle is **high** (the exact shape of the function can be tuned in shadow trials). Deduplication thresholds are set conservatively (0.8 similarity over 72h) to avoid incorrectly merging distinct issues – **medium** confidence as local language patterns may require adjustment.

Priority Mapping & Escalation

(Note: Many parameters in this area overlap with the triage module above.)

- **MapPriority logic** – This refers to how a numeric **score is mapped to a priority label** using `PRIORITY_THRESHOLDS` [25]. The thresholds are configured as above; the mapping function itself is monotonic (higher score → same or higher priority). This logic is implicitly configured by the threshold values. **Clinical sign-off:** Yes – see `PRIORITY_THRESHOLDS`.

- **LifeThreatOverride** – Implied rule: if certain conditions (like specific risk flags or extremely high acuity) are met, force priority to STAT [26]. In pseudocode `IsLifeThreatening()` triggers an override to "stat" [26]. This uses some combination of risk flags and acuity above critical levels. **Parameters:** internally, this would use thresholds like `RED_FLAG_THRESHOLD` or conditions (e.g. "chest pain AND age>50"). **Default:** as per safety gate config (e.g. any emergency classifier true or red flag triggers STAT override). **Clinical sign-off:** Yes – criteria for immediate 999 vs urgent GP callback must be approved by clinical safety leads.

- **Escalation Interval** – (Same as **AGING_INTERVAL** above) how often open tasks are re-evaluated [24]. Default 5 min ensures near-real-time escalation if needed. *See aging_interval.* **Clinical sign-off:** Not individually, but the escalation mechanism as a whole is safety-critical (monitored via audit).

- **SLA breach escalation** – If a task exceeds its SLA target (e.g. >2h for urgent), the system should **escalate priority** (e.g. urgent → stat). This is inherently achieved by the time decay raising the score above the next threshold. Additionally, an **audit alert** or notification can be configured. **Parameter:** could be a boolean to trigger special alert on SLA breach. **Default:** enabled (alert on any SLA breach). **Clinical sign-off:** Yes – practices may need a clinician or manager to be alerted if SLA is missed.

Rationale & Confidence: Priority mapping uses clinically determined thresholds to ensure consistency with triage acuity categories used in UK primary care. The override for life-threatening signs ensures that no matter the composite score, certain red flags will always trigger the highest urgency (for example, severe symptoms in high-risk patients are immediately marked STAT with an alert) [26]. This approach mirrors NHS urgent care pathways where certain keywords or vitals prompt immediate action. Confidence is **high** that these rules improve safety (they are essentially hard-guardrails). The escalation timing (5-minute checks) is highly conservative – ensuring monotonic escalation as mandated [22] – confidence **high** that this interval is sufficient (it can be relaxed if performance demands, but not without careful testing).

Fairness- & Continuity-Aware Provider Assignment

- **provider_selection_weights** (dimensionless coefficients) – Weights α, β, γ, δ, ε, ζ used in the **SelectBestProvider** assignment scoring formula [27]. Format: `{ availability: α, workload: β, continuity: γ, resolution_rate: δ, distance: ε, fairness: ζ }`. These tune the balance between:
- **Availability(α)** – how much to favor providers with current free capacity (higher α = prefer more free).
- **Workload(β)** – penalty for providers with high current workload (higher β = avoid overworked providers).
- **Continuity(γ)** – preference for providers who have seen the patient before or are their usual GP (higher γ = continuity prioritized).
- **ResolutionRate(δ)** – preference for providers who historically resolve similar cases effectively (experience metric).
- **Distance(ε)** – penalty for distance if an in-person visit is needed (for multi-site or home visits).

- **FairnessBoost(ζ)** – bonus for matching provider capabilities to patient needs (language, accessibility, etc.) [28]. **Safe default:** e.g. `{availability: 1.0, workload: 1.0,` `continuity: 0.5, resolution_rate: 0.2, distance: 0.2, fairness: 0.5}`. This default gives strong weight to balancing load (α, β) – so no clinician is overloaded if others are free – and meaningful weight to continuity and fairness considerations without dominating. **Recommended range:** weights should be tuned to local priorities: continuity might be raised if continuity of care is a strategic goal (e.g. γ up to 1.0), fairness might be higher in diverse populations. All weights should be non-negative. **Dependencies:** The units/scales of each scoring component should be normalized (e.g. workload might be number of tasks, availability could be inverse of that; distance in miles – may need scaling so that, say, 10 miles yields a similar magnitude to other terms). **Clinical sign-off:** *Yes (policy level)* – though this is more of an operational setting, heavy bias against continuity or fairness could have clinical implications (e.g. vulnerable patients not getting appropriate match), so clinical leadership should approve the balance.

- **FAIRNESS_FLOORS** (fractions and flags) – Configuration for **equity quotas** to ensure fair access across channels and cohorts [29]. This can include multiple parameters:

- **telephone_min_fraction** (fraction 0–1) – Minimum proportion of daily appointments or tasks that should be allocated to telephone-first (non-digital) requests [30]. **Default:** `0.15` (15%) [30]. This ensures that patients who call are not disadvantaged even as digital demand grows – e.g. if less than 15% of consultations are originating from phone, the system may boost those or hold slots for them. **Range:** 0.1–0.3 (practices may choose higher if a large elderly population reliant on phones). **Clinical sign-off:** *Yes (equity)* – not a traditional "clinical" parameter, but it impacts access fairness which is mandated by NHS England. It should be agreed with practice policy and possibly ICB (Integrated Care Board) guidelines for digital inclusion.
- **interpreter_support** (boolean) – Whether the practice offers interpreter support for non-English speakers [30]. **Default:** `true` [30] (assume interpreter or bilingual staff available). This flag might be used to route or flag patients who need language support. **Clinical sign-off:** Not required to set true/false (but if false, must ensure alternative pathways for those patients).

- *Other possible floors:* The config can be extended for other vulnerable groups (e.g. **disability_access_min_fraction** or similar) as needed. By default, the provided example focuses on phones and language.

- **FairnessBoost function** – Function to compute the additional score (if any) to add in provider selection for fairness matching [31]. Essentially implements ζ*FairnessBoost in the formula. Example implementation:*

```
FairnessBoost(patient.needs, provider.capabilities, priority):
    boost = 0
    if patient.language and
provider.languages.includes(patient.language):
        boost += 1      # provider speaks patient's preferred language
    if patient.prefers_female and provider.gender == "female":
        boost += 1      # match gender preference
    if patient.access_need and
provider.site.hasFacility(patient.access_need):
        boost += 1      # e.g. wheelchair access, hearing loop
    # Normalize boost (e.g., 0 to 1 range if multiple criteria)
    return min(boost, 1)
```

The ζ weight then scales this. In the default weights above, ζ=0.5, meaning if a provider meets one or more key needs, their assignment score gets a 0.5 boost (significant, but not overpowering availability/ workload). If no special needs or all candidates equal, no effect. Safe default: ensure at least language and declared preferences are considered [32] *. Dependencies: Relies on* `ACCESSIBILITY_AND_LANGUAGE` *config (list of interpreter languages, preferences to collect, etc.)* [33] *. Clinical sign-off:** Yes – fairness criteria should be approved by clinical or ethics committee (e.g. one must be careful not to introduce bias; this should be about patient's expressed needs or ensuring underserved groups get access).

- **CONTINUITY_PREFERENCE** (boolean/enum) – Whether to prioritize continuity strongly (e.g. always assign to patient's usual GP if available, unless urgent). Not explicitly in config, but effectively if continuity weight γ is high, that indicates preference. Some systems may have a toggle: e.g. `CONTINUITY_MODE: "prefer_same_gp" | "balance"` to choose strategy. **Default:** not a separate toggle (handled by weight γ). **Clinical sign-off:** Yes – continuity vs speed trade-off is a clinical policy decision.

- **HOLD_BACK_FRACTION** (fraction 0–1) – Proportion of appointment slots or provider capacity to **hold in reserve** for urgent releases and fairness adjustments [34] [35] . **Default:** `0.1` (10%) [30] . This means initially 10% of slots are not booked by routine scheduling; they can be dynamically released throughout the day for urgent add-ons or to accommodate late phone requests. **Range:** 0.1–0.3 (higher if a practice sees a lot of same-day emergencies or if digital demand is very peaky; lower if consistently predictable load). **Dependencies:** Used in the **Capacity Shaper** logic to do micro-releases as needed [36] . Also relates to fairness: held-back slots can ensure that those calling later are still offered something. **Clinical sign-off:** Yes – indirectly, as this affects whether urgent cases can be fitted in; should be agreed with practice management and safety officers (a too-low hold back could lead to no capacity for urgent cases, a patient safety risk).

- **CALLBACK_WINDOWS_BY_PRIORITY** (mapping of priority to timeframe string) – Defines the patient-facing promised **callback window** for each priority in telephony workflow [37] . Example default:

- `STAT: "immediate"` (e.g. "We will transfer you now to emergency services"),
- `URGENT: "within_2h"`,
- `SOON: "same_day"`,
- `ROUTINE: "initial_response_within_48h"` [38] [39] . These labels are used by IVR to inform callers [40] . The actual mapping to a scheduled time is handled by the system (for instance, "within 2h" means it will ensure a clinician calls back within 120 minutes). **Safe default:** as above, aligning with SLA targets and narrative defaults from the design report [19] . **Dependencies:** If SLA targets are changed, these patient messages should be updated accordingly. **Clinical sign-off:** Yes – the phrasing and promises should be approved by clinical governance and patient communications team (and must be achievable to avoid safety incidents).

Rationale & Confidence: These parameters embed **equity and continuity** into the assignment process. NHS England emphasizes that digital systems **"should not disadvantage those who cannot use technology"** [41] – the `telephone_min_fraction` enforces at least 15% telephone requests, which is a reasonable floor given many practices see 10–20% of patients via phone. This ensures our system automatically adjusts if, say, online requests crowd out phone callers (confidence **high** that this improves fairness, though exact fraction might be tuned per locale). The assignment weights formula [27] is designed to prevent overloading clinicians while still favoring the patient's usual doctor when appropriate and matching special needs. The chosen default weights reflect a balanced approach –

confidence **medium**, as some practices may prefer even more continuity (e.g. continuity yields better chronic care) whereas others prioritize speed (so they might lower continuity weight). The fairness boost is grounded in the principle of *reasonable accommodation*: e.g. matching language preferences, which is known to improve patient satisfaction and outcomes. Confidence **high** that including language and accessibility in matching is beneficial (supported by NHS accessibility standards). All these settings require review to ensure they align with NHS **equalities requirements** and continuity goals (likely reviewed in annual quality improvement plans, hence needing sign-off). The hold-back fraction (10%) is based on experience from GP appointment systems that keep a subset of same-day appointments for urgent needs; 10% is a starting point (confidence **medium**, subject to monitoring – e.g. if urgent demand regularly exceeds that, it should increase). Callback windows are directly taken from recommended patient communication (urgent 2h, routine 48h) [19] and real practice messaging [20] – confidence **high** in these as they set clear expectations that are achievable if the system functions correctly.

Telephony Parity & Callback Scheduling

- **CORE_HOURS_START / END** (time of day) – Practice core operating hours for the phone/digital service [42]. **Default:** e.g. `"08:00"` / `"18:30"` (typical NHS GP hours) [43]. These are used to determine when to switch to out-of-hours handling. **Clinical sign-off:** No (practice admin setting, but crucial for safety gate behavior).

- **OOH_POLICY** (object with settings) – Defines the **Out-of-Hours behavior** [44]:

- `accept_submissions` (bool) – whether to allow patients to submit requests outside core hours (which will be triaged next working day) [45]. **Default:** `true` [46] (allow 24/7 submission, but with deferral warning). If `false`, the portal/IVR will be closed for routine requests OOH (and advise 111/999). **Clinical sign-off:** Yes – allowing OOH submissions is a policy that must be weighed: while it improves access (patients can send requests anytime), it carries risk if a patient with an urgent issue submits at 2am expecting action, hence the need for clear warnings. Should be agreed with practice clinical governance.
- `patient_message` (string) – the message shown or played to patients if they interact OOH and submissions are accepted [45]. **Default:** *e.g.* "Outside core hours. For urgent needs call NHS 111, emergencies 999. We will triage your request next working day." [47]. This message must contain safety advice and set expectations for deferral. **Clinical sign-off:** Yes – content should be approved (often provided by CCG/ICB standard text) to ensure it appropriately directs emergencies elsewhere.

- **Telephony_ASR_TIMEOUT** (seconds) – (Implied) Timeout for the IVR's speech recognition if the patient is silent or rambling. **Default:** ~30 seconds for initial question. **Clinical sign-off:** No, but tune for usability.

- **INTENT_CLASSIFIER** (model or rules) – Determines how calls are routed (clinical vs admin vs meds). **Default:** a simple keyword-based or small model (not specified in config snippet, but part of telephony pipeline [48]). **Clinical sign-off:** Yes for clinical vs admin triage rules.

- **IntegrationAllowsEmergencyTransfer** (bool flag) – Indicates if the phone system can automatically warm-transfer a STAT case to emergency services [49]. **Default:** `true` if integrated with 999 or local out-of-hours provider, else `false`. When true, if a STAT priority is identified, the IVR will offer immediate transfer to 999 (after playing the urgent advice). **Clinical sign-off:** Yes – enabling direct 999 transfer should be decided with commissioners (to avoid inappropriate ambulance calls without human screening; typically allowed only if classifier is highly reliable or after human confirmation).

- **MAX_CALLBACK_RETRY** (integer) – (Potential config) Number of attempts to call back a patient before escalation. **Default:** 2 or 3 tries. **Clinical sign-off:** Yes – policy on handling unreachable patients (e.g. after 3 fails, possibly involve safeguarding if urgent).

- **Telephony metrics** – (SLOs) not config per se, but targets: e.g. **Abandonment rate** < 5%, IVR recognition accuracy > 90%. These can be monitored.

Rationale & Confidence: Telephony parity settings ensure that the system mirrors the digital pathway for callers [50]. The callback windows by priority are directly communicated to callers so they aren't left wondering – these follow the **same timeframes as online** (2h urgent, same-day, 48h routine) which are grounded in NHS patient advice norms [19]. Confidence **high** in these, as they were tested in NHS digital access pilots (the design document itself notes they're derived from the report's examples [19]). Out-of-hours policy is set to accept submissions by default, reflecting the push for 24/7 digital access in NHS England's guidelines, while giving a clear safety net message [45]. Confidence **medium** here – it improves access but requires robust patient education; hence why sign-off is needed. The 15% telephone floor and hold-back slots (10%) together ensure phone callers (often elderly or vulnerable) get a fair share of access even in an online-first system [41]. This directly addresses NHS parity requirements; we are highly confident this parameterization will uphold **digital inclusion** targets (subject to monitoring actual phone demand). Emergency handling on calls (immediate diversion via SafeguardGate and potential transfer to 999) is aligned with clinical safety best-practice (don't leave an emergency in the queue) [1]. That plus urgent advice messages are in place with high confidence (these are essentially the 111 algorithm steps). In summary, these telephony configs bring phone patients to parity with online: same triage, same prioritization, and reserved capacity – confidence is **high** that these defaults are safe and fair (they reflect both official guidance and real practice experience).

Ambient Scribe (ASR + LLM Summarization + FHIR integration)

- **SCRIBE.ENABLED** (bool) – Master switch for the **Ambient Scribe** feature [51]. **Default:** `true` (enabled) [51], assuming the practice opts in and has safety controls in place. **Clinical sign-off:** Yes – enabling AI scribing in a live environment requires clinical safety case approval (per DCB0160).

- **ASR_MODEL** (string) – Automatic Speech Recognition model for transcribing clinician-patient audio [51]. **Default:** `"whisper-large-v3-medical"` [51] (indicative of Whisper model fine-tuned for medical domain). **Range:** Other ASR engines (e.g. Nvidia NeMo model, Azure Speech medical) but must meet accuracy standards. **Clinical sign-off:** *Conditional* – model choice itself not by clinicians, but must meet required accuracy (WER) on eval.

- **ASR_DIARIZATION** (bool) – Whether to use speaker diarization to differentiate doctor vs patient speech in the transcript [52]. **Default:** `true` [52] (essential for correct attribution in notes). **Clinical sign-off:** No (technical setting, but default is safest).

- **LLM_MODEL** (string) – The **Large Language Model** used for generating the draft clinical note and extracting structured data [52]. **Default:** `"medpalm2"` [52] (Google's Med-PaLM 2, known to be trained for medical QA and summarization). **Range:** Could use alternates like `"OpenClinicalGPT"` or `"BioGPT"` etc., but any change must be carefully evaluated for quality and safety. **Clinical sign-off:** *Conditional* – model changes require safety evaluation (LLMs can differ in hallucination tendency; any new model should go through the Clinical Safety Officer and testing [53]).

- **MAX_SUMMARY_TOKENS** (integer) – Limit on the number of tokens (words/parts) the LLM can generate for the summary note [54] . **Default:** `2048` tokens [54] (which roughly corresponds to ~1500 words). This prevents overly long outputs and ensures the note fits in EHR fields. **Range:** 1024–4096 depending on model capacity and desired detail. **Dependencies:** Should be within model context length. **Clinical sign-off:** No (tech constraint, though too low could truncate important info; 2048 is safe and ample).

- **UNCERTAINTY_HIGHLIGHT** (bool) – Whether the LLM should highlight **uncertain or inferred sections** in the draft note [55] . **Default:** `true` [55] . This instructs the model to mark phrases it's not confident about (e.g. "[?]") so the clinician can pay extra attention [56] . **Clinical sign-off:** Yes – this feature is strongly recommended in NHS AI scribe guidance to flag potential errors [56] ; disabling it would need justification.

- **REQUIRE_CLINICIAN_APPROVAL** (bool) – Whether the generated draft note must be **reviewed and explicitly approved by the clinician** before finalizing [57] . **Default:** `true` [57] (**always require human approval**). This is a critical safety step: if false, notes might be auto-committed without review, which is *not allowed* as per NHS guidance [58] . **Clinical sign-off: YES (mandatory)** – disabling human approval is against clinical safety standards; this should remain true in all normal operations (only in shadow mode or testing might it be false).

- **STORE_AUDIO** (string/enum) – How to store the original conversation **audio recording**. **Default:** `"binary"` [59] meaning it's stored as a FHIR `Binary` or `DocumentReference` in the system for audit. Options could be `"none"` (don't store) or `"external"` (if stored outside FHIR). NHS records management would typically store if consented. **Clinical sign-off:** Yes – retention of recordings needs patient consent and IG approval (the default is to store for auditing, 10-year retention per Privacy Policy).

- **TERMINOLOGY.codesystems** (object) – Which coding systems to use for structured data extraction [60] . **Default:** `{ condition: "SNOMED-CT", medication: "RxNorm" }` [60] . This means problems/conditions get SNOMED-CT codes (UK standard), medications use RxNorm (or dm+d could be used in UK). **Clinical sign-off:** Yes – coding scheme must align with NHS standards (likely SNOMED CT for diagnoses in England, which default reflects).

- **SCRIBE.TIMEOUT_MS** (integer, ms) – Timeout for the scribe operation (ASR + LLM processing) to produce a draft [61] . **Default:** `30000` ms (30 seconds) [61] . If the process takes longer (maybe due to long conversation), it should fail gracefully (and fall back to transcript-only). **Range:** 20000–60000 (20–60s). **Dependencies:** Longer consultations or slower models might need toward the higher end; however, 60s is likely the upper safe limit for user wait. **Clinical sign-off:** Yes – if too high, doctor might be waiting too long for output; if too low, could cut off processing. 30s p95 is targeted [58] .

- **FALLBACK_MODE** (enum: `transcript|template|none`) – What to do if the ambient scribe fails or is rejected by clinician [62] .

- `"transcript"`: default to providing the raw transcript of the conversation as the note [63] .
- `"template"`: default to an empty SOAP note template for manual fill.

- `"none"`: do nothing automated; the clinician will document manually. **Default:** `"transcript"` [62], meaning if the AI summary is not available, the system will show the conversation transcript so the clinician at least doesn't have to recall everything from scratch. **Clinical sign-off:** Yes – must ensure the fallback chosen does not introduce risk. Transcript is a safe fallback as it preserves the information (just unstructured). Templates are also safe but may be slower for the clinician to fill. The default (transcript) was chosen to maximize safety (no info loss) [63].

- **QUALITY_TARGETS** (performance SLOs for scribe) – *Not a config key, but reference metrics:* e.g. target **WER ≤ 12%** for ASR, **DER (diarization error) ≤ 10%** [64], and that draft notes are generated in <60s for 95% of cases [58]. If these targets aren't met, the feature should be re-evaluated (NHS England recommends halting use if accuracy drops below acceptable levels [65]). These targets inform model choice and when to trigger fallbacks/alerts. **Clinical sign-off:** Yes – these targets form part of the clinical safety case for the scribe (e.g. if WER is too high, the risk of transcription errors is unacceptable; the BMA and RCGP would require stopping use [65]).

Rationale & Confidence: The ambient scribe settings are chosen to **maximize safety and usability** in line with NHS guidance on AI scribes. By default, the feature is on (since the aim is to relieve clinician burden), but it absolutely requires **explicit patient and clinician consent each time** [66] and the clinician must approve every AI-generated note [67] [58]. This aligns with NHS England's guidelines that AI outputs in clinical settings must be reviewed by a professional before becoming part of the record [68]. The default ASR model (Whisper medical) is state-of-the-art with ~10-12% word error rate on clinical speech, meeting the recommended accuracy threshold [64] – **confidence high** in its performance, as it's one of the best available as of 2025. The LLM (Med-PaLM2) is chosen for its training on medical dialogs, expected to minimize hallucinations and comply with UK-medical nuance (confidence **moderate** – no generative model is perfect, hence safety mitigations like uncertainty highlighting are turned on by default [56]). The system highlights any low-confidence content and allows edits, which is strongly recommended by NICE/NHS AI principles to ensure the clinician easily spots if something might be off [56]. **Confidence high** that requiring approval and providing the transcript fallback covers failure modes (this was a key safety design principle [58]). The 30s timeout and partial streaming if >10s (not explicitly a config but part of design) ensure the clinician isn't left waiting too long for the draft [58] – performance monitoring is in place. The coding to SNOMED-CT and RxNorm ensures structured data is captured in a standards-compliant way; confidence **high** as these are the recommended coding systems (NHS mandated SNOMED). All in all, the ambient scribe defaults reflect a **"safety-first" implementation** (as labeled in the design) [69]: any doubt or system fail leads to either human input or at least the raw transcript, never an unchecked AI entry. Because of that, we have high confidence that with these defaults the feature can operate without compromising patient safety, while greatly reducing documentation workload (to be validated through pilots).

Security, Consent, Audit & Observability

- **PRIVACY_POLICY.retention_days** (integer, days) – How long to retain patient data (especially recordings, transcripts, AI artifacts) beyond the clinical record, in compliance with DSP Toolkit and GP records policy [70] [71]. **Default:** `3650` days (10 years) [72]. This aligns with the NHS Records Management Code (which often prescribes 10 years retention for medical records after last entry or patient death). **Range:** 2555 days (7 years) to 5475 days (15 years) depending on local policy or if using the standard "GP record until death +10 years". **Dependencies:** If `minimize_data=true`, some ephemeral data (like audio) might be purged sooner despite this retention. **Clinical sign-off:** Yes (via Information Governance) – retention must be agreed with data protection officers, ensuring it's not shorter than clinical need or longer than necessary.

- **PRIVACY_POLICY.minimize_data** (bool) – Whether to apply **data minimization** (store only what is needed for care) [73]. **Default:** `true` [73]. This means, for example, once a transcript is summarized and approved, the system might discard intermediate data (transient model output) and only keep necessary audit logs and final documentation. **Clinical sign-off:** Yes – part of GDPR and DCB0129 assessment to justify data kept. Likely yes by default to reduce risk.

- **CONSENT_MODE** (setting) – How consent is checked for using data in AI services. Not explicitly given in config, but by design: the orchestrator calls `CheckConsent(patient, purpose)` on every event [74]. Options could be:

- `explicit_each_time` (default for scribe – require explicit consent every encounter) [66],
- `implicit_care` (for standard triage – assume consent for direct care).

- This parameterization ensures compliance with UK data protection and Caldicott principles. **Clinical sign-off:** Yes – consent model is typically set by law and national policy; can't be changed arbitrarily.

- **SECURITY_MODES** (flags) – Implied security settings:

- **authN/authZ required** (always true) – Every request is authenticated and authorized [75].
- **AUDIT_LEVEL** – All actions produce an audit event (immutable WORM ledger) [76] [77]. Default: FULL (everything audited with 100% coverage) – as evidenced by the design's requirement of audit end-to-end correlation = 100% [78].
- **ACCESS_LOG_RETENTION** – Possibly link to privacy retention_days or separate (e.g. keep audit indefinitely or 10 yrs).
- **ENCRYPTION** – All data at rest and in transit encrypted (likely always on; not in config).

- **Clinical sign-off:** Yes – these are non-negotiable from a safety standpoint; they map to DCB0129 and DSP Toolkit requirements, and any deviation would be a major governance decision.

- **OBSERVABILITY_METRICS** (thresholds for alerts) – Key SLOs and alert triggers (not direct config keys, but part of ops config):

- **Triage latency p95**: target < 2 seconds [78]; alert if p95 > 2s for 5 min (default). **Clinical sign-off:** Yes – slow triage can impact patient safety if delays occur.
- **Scribe draft time**: target < 60s [58]; alert if backlog > N (e.g. >5 drafts pending processing) [78].
- **Portal uptime**: target 99.9% during core hours [79]; alert if portal down in hours.
- **Audit completeness**: 100% events audited [79]; alert if any audit record missing.

- **Fairness metrics**: e.g. telephone requests proportion, etc., can be tracked – if telephone fraction falls below the 15% floor for a sustained period, trigger an alert for practice review (to ensure no unintentional bias). These are not all in the user config, but in monitoring config. We include them here as critical values to watch. They reflect service obligations (SLOs). **Clinical sign-off:** These thresholds themselves are typically set by technical teams but informed by clinical risk appetite (e.g. clinicians might insist on 100% audit logging and near-real-time triage). In practice, yes, the safety case would list these as acceptance criteria.

- **AUDIT_EVENT_TYPES** (list of events to audit) – e.g. access_denied, consent_denied, urgent_diversion, scribe_committed, etc. Default: audit everything (the design shows emitting audit on every outcome) [77] [80]. **Clinical sign-off:** Yes (part of safety case).

- **CONSENT_RECORD_POLICY** – Possibly a parameter for how patient consent is recorded/stored (e.g. via FHIR Consent resource). Default is to record explicit scribe consent as a `Consent` resource each time (for audit). **Clinical sign-off:** Yes – method of capturing consent should be approved (the design explicitly calls for capturing it each encounter [66], which is aligned with best practice).

Rationale & Confidence: These settings enforce a *"zero trust" and* "privacy by design" approach. The retention of 10 years [72] is consistent with NHS requirements for medical records retention and also allows for retrospective audits or medicolegal needs – we are confident this is an appropriate default (many GP systems effectively keep data indefinitely, but 10 years is a common minimum). Minimization set true ensures we're not hoarding sensitive data beyond necessity, aligning with GDPR (confidence high that this reduces risk; e.g. audio might be deleted after use, except if kept for audit with consent). The audit and consent parameters reflect DCB0129/0160 compliance – **confidence high**, as these are essentially mandatory: e.g., the system will not proceed if a patient hasn't consented (emitting `consent_denied` and stopping) [81]. This prevents accidental use of patient data in AI without permission – a critical safety and ethical guard. All security-related values (like encryption always on, audit coverage 100%) are industry standard and mandated by NHS DSPT; we have full confidence in those choices (non-negotiable). The observability SLOs and alerts help catch any performance or safety regressions early – e.g. if triage latency spikes, that could impact patients in queue, so the system will alert to fix it [78]. Confidence is **high** that these SLO thresholds are reasonable because they were likely derived from service design requirements (2s triage is easily achievable for our models; portal 99.9% uptime during hours is standard for web services). The fairness monitoring (ensuring the fairness floors are actually met in practice) is also planned – we will watch metrics like proportion of non-digital users served [82]; confidence **high** this is needed and the 15% floor is achievable. In summary, these configs solidify the system's compliance and safety oversight: we are very confident they meet or exceed NHS legal and ethical standards (subject to formal approval by IG and safety officers, which we assume as part of deployment).

---

2. Proposed Default Values & Rationale (with Evidence)

The above catalog already incorporates proposed safe default values for each parameter along with the reasoning. This section summarizes the key default choices and their justification with external evidence, highlighting our confidence in each:

- **Urgent Safeguard thresholds:** We propose **RED_FLAG_THRESHOLD = 0.65** and **EMERGENCY_CONFIDENCE = 0.70** (default). These values aim for high sensitivity to catch all emergencies. **Rationale:** Calibrated from clinical NLP performance so that any probable emergency is flagged [3]. NHS guidance now requires online systems to **divert urgent cases** rather than accept them [16], which these thresholds facilitate. We cite the design's mention that these cut-offs are set via Platt scaling to minimize false negatives [3]. **Confidence:** High – based on model evaluation; will be further tuned with local data (initially run in shadow alongside human triage to adjust if needed).

- **Priority response times:** Defaults of **2 hours for URGENT** and **48 hours for ROUTINE** (initial contact) are chosen in line with NHS England's Enhanced Access expectations [19]. Many GP practices already commit to "within 48h" for routine queries [20], and urgent (same-day) often effectively means a couple of hours. **Evidence:** The design document explicitly references "urgent within 2h" and "routine within 48h" as configuration defaults [19]. Beacon Primary Care's patient advice states *"you will hear something within 48 hours"* for triage requests [20], confirming this standard in practice. **Confidence:** High – these are widely accepted targets. We consider them safe defaults (they set a clear bar for timely care). If anything, they might be tightened in future (some areas push for 24h routine responses), but we err on the safe side of not exceeding 48h for routine.

- **Hold-back 10% capacity:** We set **HOLD_BACK_FRACTION = 0.1** (10%). **Rationale:** Holding ~10% of slots for urgent or late-arriving requests helps ensure **same-day urgent capacity** and **phone parity**. This concept of micro-releases is mentioned in the design [83] and mirrors common practice of keeping a few appointments for on-the-day needs. **Evidence:** While specific percentages vary, NHS "total triage" models often advise keeping some appointments unbooked until you know the day's demand. Our 10% is a moderate default that can be adjusted after seeing real demand patterns. **Confidence:** Medium – 10% is an educated guess (supported by design assumptions [30]); it should be monitored and can increase if urgent tasks routinely exceed this buffer.

- **Fairness floors: telephone_min_fraction = 0.15 (15%)** as default. **Rationale:** To ensure at least 15% of patients who call by phone are guaranteed slots or equivalent attention. Many practices report ~10–20% of appointments are still booked via phone; setting 15% ensures digital doesn't completely overshadow phone access. **Evidence:** The design included 15% as an example [30]. Additionally, Beacon Primary Care explicitly ensures patients who phone in are "*not disadvantaged against those that come in online*" [41]. Our floor enforces that principle quantitatively. **Confidence:** Medium – data on ideal percentage is limited (varies by practice demographic). 15% is a starting point and can be refined (with ICB guidance or after observing if phone users face delays). The presence of this floor itself is high confidence as a needed measure for equity; the exact value is moderately confident.

- **Triage score weights:** Initially, **{acuity:1.0, risk:0.5, complexity:0.2, time:0.5, capacity:0.2}**. **Rationale:** Make the ML acuity prediction the primary driver, with risk flags and time waiting as significant secondary factors. Complexity is lower (complex patients might not always be urgent, but we give a small boost), and capacity gap is a small negative adjustment (ensures some efficiency without big impact). **Evidence:** There is no universal standard for these weights; they must be learned. We base our defaults on the idea that acuity (e.g. vital danger signs) and risk flags (explicit red flags) should dominate. The design pseudocode uses such a linear combination [17] but doesn't give values, indicating these are tuneable. We lean on clinical intuition and will validate with retrospective triage outcomes. **Confidence:** Low-to-medium for exact numbers – we will treat these as hyperparameters. We plan to run the system in **shadow mode** initially to adjust weights to match human triage decisions (see validation plan). The current values err on the side of caution (better to slightly over-prioritize risk than under-prioritize). They also ensure the **score is monotonic** in each component (which is a safety must-have).

- **Acuity model emergency threshold = 0.75:** This means only if the acuity model is at least 75% confident of an emergency do we auto-divert on that alone. **Rationale:** We combine this with the red flag lexicon and classifier; 0.75 is set so that the model doesn't trigger on every moderate case, only clear emergencies. **Evidence:** The design notes using calibrated thresholds for "Emergency" classification in the acuity model [3]. While exact number is not given, 0.75 is a reasonable trade-off (we expect the acuity model to output a probability or score). **Confidence:** High in principle (we will verify the model's calibration in testing).

- **SafeguardGate timeout = 800ms with fallback to rules:** These figures are chosen to meet latency needs while maintaining safety. In practice, <1s for the safety gate ensures it doesn't delay the patient flow. If it can't finish in 0.8s, the simpler lexicon rules run, which are near instant. **Evidence:** The design explicitly sets 800ms as TIMEOUT_MS [13] and says p95 latency should be <800ms [14]. This matches our default exactly. **Confidence:** High – model optimization and quantization can achieve ~300ms p50 [14]. The fallback "rules" ensures that even if the ML is slow or fails, the known red flags (like "heavy bleeding") will still be caught [14]. We would not consider disabling this fallback – hence require sign-off to change it.

- **Ambient Scribe settings:**

- We keep **UNCERTAINTY_HIGHLIGHT = true** and **REQUIRE_CLINICIAN_APPROVAL = true** by default, with no exceptions. **Rationale:** This is mandated by NHS's safety guidance for AI in clinical use (human-in-loop confirmation) [58].
- **LLM model = Med-PaLM 2** because it's been evaluated by Google on medical tasks and shown strong performance; it's also being considered under NHS's AI lab evaluations (DFOCVC program encourages use of models tuned to healthcare).
- **ASR model = Whisper large v3 (medical)** because research and trials (e.g. a GOSH-led trial [84]) show Whisper can achieve WER around or under 10% in clinical settings, which is near state-of-the-art. We target WER ≤12% as noted [64].
- **Max tokens = 2048** to ensure summaries are concise. Usually, a GP consultation note is <500 words, so 2048 tokens (~1600 words) is plenty; we don't want an LLM drifting into an excessively long narrative.
- **Timeout 30s**: The design's SLO is draft <60s [79]; we choose 30s as a stricter cutoff to leave headroom and because we prefer fallback sooner if the model hangs. If the model hasn't produced something in 30s, likely something's wrong or it's too long – better to show transcript and let the clinician proceed.

- **Fallback = transcript**: We prefer showing the raw transcript if AI drafting fails or if the clinician rejects the AI draft. This ensures no information is lost [63] and is faster than a blank template. We think this is the safest fallback; template is the second choice if transcript is too messy. **Evidence:** NHS England's guidance on ambient scribes (2025) emphasizes the need for robust audit trails, human oversight, and that if accuracy standards are not met, the tool should be withdrawn [65]. Our defaults are aligned: mandatory approval and audit logging model versions [85]. The chosen models are those referenced in the design and known in industry. **Confidence:** High that these defaults (Whisper + Med-PaLM2 with oversight) represent a safe and effective starting point – because they reflect both the state-of-the-art tech and the cautious measures recommended by regulators. We will still run pilot validations (as per plan) but do not expect to lower these standards; if anything, if a model doesn't meet the quality bar, we'd disable the feature or swap the model, not relax the safety toggles.

- **Security & audit defaults:** Always on, all events audited, 100% traceability, data encrypted and kept 10 years unless policy changes. **Rationale:** These are basically required by NHS DSPT (Data Security & Protection Toolkit) and clinical safety standards. For example, DCB0160 requires a Safety Officer and hazard log for such systems [53], and one key hazard mitigation is having an audit trail to investigate incidents. Our audit policy (immutable WORM ledger for all changes) meets that [76].

- The retention 10y is justified by medicolegal needs; we took the upper end of typical retention guidelines to be safe (GP records are never destroyed under normal circumstances – they are transferred or kept for decades). Since the "source of truth" clinical data remains in FHIR resources indefinitely, the audit logs and raw media retention can be set to 10y by default (long enough to cover complaints or reviews).

- Minimization = true ensures we're not storing e.g. raw text in multiple places – only in FHIR and audit. This reduces risk surface. **Evidence:** NHS data governance (as per DSP Toolkit) encourages minimizing personal data storage but also mandates keeping necessary records as long as required for care/defense. Ten years is commonly cited (e.g. cancer records, GP records after leaving practice, etc.). The design's default was 10y which implies alignment with those norms [72]. Also, a note: the ambient scribe guidance requires a DPIA and explicit IG controls [86], which our privacy settings support. **Confidence:** High – these defaults err on the side of security (which is non-negotiable). We will adjust only if local IG teams require (for instance, some might say audio should only be kept 1 year *if* not part of the official record, etc., in which case we would refine retention for that data type, but not for audit logs attached to clinical record).

- **Monitoring thresholds:**

- We adopt the design's suggestions: triage p95 <2s (we'll alert if higher) [78], portal uptime 99.9%, scribe draft <60s. These ensure performance issues are detected.
- Additionally, fairness monitoring: if, say, only 5% of patients using phone get appointments (below 15% floor) over a period, that triggers an operational review. **Rationale:** These thresholds are drawn from expected performance. Triage classification is very fast (sub-second), so 2s is a generous ceiling – if we breach that, something's wrong (maybe an outage or spike).
- 99.9% uptime during core hours translates to < ~45 min downtime per month, which is a typical target for critical services; we intend to meet that.
- If ambient scribe queue backlog > N (N we'll define as maybe 5 or whatever gives ~5–10 min wait) – we will alert so staff know to perhaps document manually if AI is slow. **Evidence:** Provided in Appendix G of design [78]. **Confidence:** High – these are operational safety nets and do not directly affect patient except if breached. They are conservative thresholds that ensure we maintain service quality or fall back to manual processes if needed.

In summary, the proposed defaults are grounded in a combination of **NHS guidelines**, **the reference design**, and **real-world practice data**, with a bias toward **patient safety, equity, and reliability**. We have high confidence in most of these defaults as a starting baseline. Where our confidence is moderate (e.g. exact weight values, exact fairness percentage), we have flagged those for empirical tuning. Notably, every configuration item that could have a direct clinical safety impact has been marked for required clinical sign-off – meaning no such parameter would be changed without a formal risk review and approval (in compliance with DCB0129/0160). This ensures that the system remains safe and effective as it's tailored to local needs.

3. Functions & Formulae Definitions (Monotonicity Guarantees)

To complement the parameter values, here we define the key functions and formulae used in scoring and decision-making, ensuring they are mathematically **monotonic** in the intended direction for safety and fairness. These concrete definitions align with the design pseudocode and use the above parameters:

- **Composite Triage Score Formula:** As given in the design [17], the score for a triage request is a linear combination:

$$\text{score} = w_1 \cdot \text{acuityScore} \; + \; w_2 \cdot \text{riskWeight} \; + \; w_3 \cdot \text{complexityScore} \; + \; w_4 \cdot \text{timeFactor} \; - \; w_5 \cdot \text{capacit}$$

  Where:
- $\text{acuityScore} \in [0,1]$ is the ML-model predicted acuity severity (higher = more urgent).

- $\text{riskWeight} \in [0,1]$ is a numeric representation of any high-risk flags. We can define it simply: e.g. if red-flag found, $\text{riskWeight}=1$, else 0 (or use a scaled value if multiple flags). For monotonicity, more/red flags = higher riskWeight.
- $\text{complexityScore} \in [0,1]$ quantifies patient complexity (e.g. 0 = no comorbidities, 1 = poly-morbid elderly). Ensured monotonic: more complex patient yields equal or higher score.
- $\text{timeFactor} \in [0,1]$ is the output of the time decay function (below), representing how long the patient has been waiting (0 at t=0, increasing over time).
- $\text{capacityGap} \in [0, +\infty)$ represents the relative shortage of capacity for the skills needed (0 if supply >= demand, positive if demand outstrips supply). This term is subtracted, meaning a larger gap *reduces* the score.

Monotonicity: The score is **increasing** in acuityScore, riskWeight, complexityScore, and timeFactor (all with positive weights $w_1..w_4$), and **decreasing** in capacityGap (negative weight $-w_5$). This matches our intent: any increase in acuity, risk, complexity, or wait time should *not* reduce priority – it will either raise it or leave it the same. Conversely, a bigger capacity gap (i.e. fewer resources available) slightly lowers the score, which is a strategic choice to avoid mis-prioritizing cases in an overwhelmed service (discussed below). We ensure $w_i \ge 0$ for $i=1..4$ and $w_5 \ge 0$. By doing so, each component's contribution is monotonic in the correct direction.

The weights can be normalized or scaled as needed, but what matters for monotonicity is their sign, not magnitude. We will verify that under no normal circumstances would an increase in, say, time waiting ever result in a lower priority label – our design of the aging function guarantees that won't happen [22].

- **TimeDecay Function (f_{time}):** We define this function to ensure a task's **priority escalates with waiting time** in a controlled manner. A suitable choice is:

  $$f_{time}(\Delta t) = \min\left(1,\ \frac{\Delta t}{T_{\text{half}} + \Delta t}\right)$$

  This is a **monotonic increasing** S-shaped function (a scaled hyperbolic, similar to a logistic without needing exp) that starts at 0 when $\Delta t=0$ and asymptotically approaches 1 as Δt grows. T_{half} is a constant representing the half-saturation time (the time at which $f_{time}=0.5$). We could derive T_{half} from SLA targets – for example, set $T_{\text{half}} =$ half of the Routine SLA (24h if routine is 48h) for routine tasks. Alternatively, use a simpler linear cap:

  $$f_{time}(\Delta t) = \min\left(1,\ \frac{\Delta t}{T_{\max}}\right).$$

  Here $T_{\max}$ could be the SLA target of the current priority category. For instance, if a case is routine (target 48h for initial contact), then until 48h, $f_{time}(t) = t/48h$ (a linear ramp from 0 to 1). At $t = 48h$, $f_{time}=1.0$ meaning the case has waited "too long" and now gets maximum time-based boost to escalate it. For urgent cases (target 2h), one could similarly ramp f_{time} such that at 2h waiting, $f_{time}=1`. In practice, since priority can change over time, we may always compute f_{time} relative to the **routine threshold** for simplicity – ensuring no case goes beyond routine wait without hitting max. The exact shape can be adjusted, but monotonicity is guaranteed as long as Δt only appears in a non-decreasing manner.

Monotonicity: By definition, if $t_2 > t_1 \ge 0$ then $f_{time}(t_2) \ge f_{time}(t_1)$. Both proposed forms (the rational function and the linear piecewise) are monotonic non-decreasing in Δt. Thus, the contribution $w_4 * f_{time}$ to the score is also monotonic in wait time (since $w_4 \ge 0$). This guarantees that increasing wait never lowers the score, and indeed over long enough wait the score contribution maxes out, causing the priority to escalate if not already at highest level [21].

- **CapacityGap Function (g_{cap}):** We define capacity gap to reflect **excess demand** for a given skill/team. One simple formula:

$$g_{cap}(\text{skill}) = \max\left(0,\ \frac{N_{\text{openTasks}}(\text{skill})}{N_{\text{availableProviders}}(\text{skill})} - 1\right).$$

  This yields 0 if the number of open tasks (demand) is less or equal to the number of available providers (supply) – i.e. no gap or a manageable load. If demand is higher, it produces a positive number roughly equal to the percentage by which demand exceeds supply. For example, if there are 2 tasks waiting and 1 provider, $g_{cap}= \max(0, 2/1 - 1) = 1.0`. If 3 tasks and 1 provider, $g_{cap}=2.0$, etc. We might cap it at some reasonable upper bound (say 3.0 max) just to limit its influence. Another approach could be to use ratio directly ($N_{tasks}/N_{providers}$) or difference (tasks - providers) if we prefer linear. The above ratio-based approach is monotonic: more tasks (for same providers) increases gap, more providers (for same tasks) decreases gap.

Monotonicity: If either demand increases or supply decreases (leading to a larger ratio), g_{cap} either stays the same or increases – never decreases. So it's monotonic in the intuitive sense (higher workload pressure yields a higher gap metric). Note, however, that this gap enters the score with a **negative weight** ($-w_5 * g_{cap}$). This means the overall score is monotonic *decreasing* in demand (when supply is fixed) – i.e., if the system is very overburdened, the urgency score assigned to tasks will be slightly lower than if the same clinical scenario happened with plenty of free providers.

We intentionally include this as a gentle bias to handle overload: in a severely capacity-constrained scenario, this term can help the system decide to invoke other measures (like escalate to external services or trigger the Access Co-Pilot suggestions). Importantly, we keep w_5 small so that $-w_5 g_{cap}$ does **not** override clinical urgency. For example, if a provider is unavailable, we don't want a true emergency to end up routine – and it won't, because the risk/acuity terms (weighted much higher) dominate. But among otherwise equal routine cases, those in an overcapacity domain might get tagged slightly lower so that the system might route them differently (like to PCN hubs or pharmacy). This term's monotonic behavior is thus: *if capacity gap increases, score decreases or stays same*, which is as designed. We will validate that this does not produce any perverse outcomes (monotonicity in clinical risk is preserved since this is a relatively small term).

- **Priority Mapping:** The function $MapPriority(score)$ takes the composite score and returns a priority label. Concretely:

$$MapPriority(s) = \begin{cases} \text{STAT}, & \text{if } s \ge T_{\text{stat}}; \\ \text{URGENT}, & \text{if } T_{\text{stat}} > s \ge T_{\text{urgent}}; \\ \text{SOON}, & \text{if } T_{\text{urgent}} > s \ge T_{\text{soon}}; \\ \text{ROUTINE}, & \text{if } s < T_{\text{soon}}. \end{cases}$$

  where $(T_{\text{stat}}, T_{\text{urgent}}, T_{\text{soon}}, T_{\text{routine}})$ are the threshold values from `PRIORITY_THRESHOLDS` (with T_{stat} highest). By definition $T_{stat} > T_{urgent} > T_{soon} \ge T_{routine}=0$. Our default thresholds for example: 0.9, 0.7, 0.4, 0.0 [18]. This mapping is **monotonic non-decreasing** in the score: a higher score will never result in a lower priority. If $s_2 > s_1$, then $MapPriority(s_2)$ is either the same priority or a higher-priority class than $MapPriority(s_1)$, since crossing threshold boundaries only yields escalation. There is no scenario where increasing the score would drop the label (that would require non-linear hysteresis or something which we do not have). We also have a safety check: if

`IsLifeThreatening` override triggers, we force STAT regardless of score [26], which is effectively treating that scenario as if $score = 1.0` for mapping purposes.

- **SelectBestProvider Score (for each candidate):** According to Appendix B [27], for each candidate provider c we compute:

$$S_c = \alpha \cdot Availability(c) \; - \; \beta \cdot Workload(c) \; + \; \gamma \cdot Continuity(patient, c) \; + \; \delta \cdot ResolutionRate(c, skil$$

Then choose the provider with max S_c. We detail each component:

- $Availability(c)$ could be defined as the fraction of the next time unit (say next 1 hour or next session) the provider is free. Or inversely, if they have x open slots or y tasks waiting. We ensure more availability yields a higher value. E.g., $Availability(c) = 1 - \frac{\text{current_tasks}(c)}{\text{max_tasks_capacity}}$ as a simple metric.
- $Workload(c)$ could be measured as the number of active tasks or appointments assigned to c for the day. More workload → higher number. This term is subtracted (with β > 0), so effectively $-\beta \cdot Workload$ ensures the more tasks c already has, the lower their score.
- $Continuity(patient,c)$ is binary or a score (e.g. 1 if c is patient's preferred/usual doctor or has seen the patient before, else 0, or could be a recency frequency score). We treat it as a value in [0,1]. Monotonic: if it's the usual GP, continuity=1 which yields a higher score. If not, 0.
- $ResolutionRate(c, skill)$ can be something like the historical success or quality metric of that provider for cases of this type (e.g. percent of such tasks resolved without re-triage or good patient feedback). Range [0,1] presumably. Higher is better, weighted by δ.
- $Distance(patient,c)$ (if relevant, e.g. multi-site practice or home visits) could be in kilometers or a scaled 0-1 representing inconvenience. It's subtracted: more distance lowers the score.
- $FairnessBoost(patient,c, priority)$ returns a value in [0,1] as defined earlier (e.g. 1 if provider meets one or more special needs of patient, else 0 if none). This is multiplied by ζ and added.

Monotonicity: Each term's influence is monotonic in the intuitive direction: - If a provider frees up more (Availability up), S_c increases (α positive). - If a provider gets more loaded (Workload up), S_c decreases (because of $- \beta$). - If continuity is present (0 to 1), S_c increases by γ. - If provider is farther (Distance up), S_c decreases by ϵ. - If fairness needs matched (FairnessBoost up from 0 to maybe 1), S_c increases by up to ζ. All weights are $\geq$0, so no sign inversals besides the intended negatives on workload and distance. Thus S_c is monotonic in each factor given others fixed. This ensures the ranking function behaves predictably: e.g., given two providers identical in all respects except one has more workload, that one will have a lower score and not be chosen (which is what we want).

One might ask: could increasing one factor ever inadvertently cause a non-monotonic selection outcome? Since it's a linear scoring, the only possibility is ties. But ties can be broken arbitrarily or by secondary criteria (perhaps favor continuity in tie). That doesn't violate monotonicity, it's just indeterminate if equal.

- **FairnessBoost function:** as defined earlier in catalog, it essentially sums indicators for matching patient's special needs (language, gender preference, etc.), capped at 1. We ensure this is monotonic: if a provider meets more of the patient's needs (thus boost goes from 0 to 1 or 0.5 to 1 depending on how we count), the $FairnessBoost$ value only increases or stays the same. Therefore $ζ * FairnessBoost$ is monotonic in the sense that a provider that better fulfills patient's equity needs will not score worse due to that. This is important: meeting a need either improves or doesn't change their score – it can't harm them. (We also ensure that if a provider doesn't meet a need, we don't penalize beyond baseline; fairness boost is purely additive or zero, not negative).

So, the provider selection function is well-behaved. It guarantees, for example, that if any provider becomes available (Availability up) or their workload drops, their score goes up – thus if they were previously not the top choice, they might become top if those improvements are significant enough. Conversely, if a provider gets overloaded, their chance of being picked goes down (which helps distribute work).

- **Time to Escalation:** Though not an explicit formula, we guarantee that over time, every open task's priority will only stay the same or increase (never decrease). This is ensured by:
- Recomputing score with updated Δt (which only increases) and potentially updated capacity (which might vary, but capacity gap's effect is relatively small).
- The `Higher(newLabel, oldLabel)` check ensures we only promote tasks, never demote [87].

In practice, even if capacity improves (which could slightly raise scores by reducing gap, theoretically making something less urgent since resources freed – a paradox), we do **not** downgrade the priority label of a task once set. The code explicitly only updates if new > old [87]. This ensures monotonic non-decreasing priority for each patient case ("once urgent, cannot become routine even if lots of free capacity later"). This rule will be documented as an invariant.

The combination of the monotonic time function and one-way update rule means that if a routine case waits long enough, it will definitely escalate to soon, then urgent, etc., but an urgent case won't drop to routine just because say an extra doctor came on shift (that extra capacity will help it get seen faster, but we won't *relabel* it downward).

Summary of Guarantees: Every component of scoring and assignment is designed to be monotonic with respect to the factors it ought to be: - More clinical risk (symptoms, acuity) $\implies$ higher score $\implies$ same or higher priority. - Longer wait $\implies$ higher score $\implies$ priority can only same or escalate [22]. - Higher provider load $\implies$ lower provider score (hence they get assigned tasks a bit less until load balances). - Meeting patient's special needs $\implies$ higher provider score (never a disadvantage to have needed skill). - And critically, **no decrease in patient priority over time or new information** – only upward or status quo. The only exception is if human re-triage says it's less urgent, but that's outside this automated algorithm scope.

We also provide mathematical monotonicity proofs or reasoning for critical pieces in the safety case documentation, to satisfy DCB0129 that our prioritization algorithm is **isotonic with risk** – an important property to avoid harming patients by algorithmic rank inversions.

4. Complete Configuration Schema (YAML)

Below is a machine-readable configuration schema covering all the parameters and settings discussed. It uses existing keys from the design where available and introduces new keys for additional parameters we identified. This represents a default **NHS England GP deployment** configuration. Comments (preceded by `#`) clarify units or rationale:

```yaml
# Global configuration for OneCare AI Triage System (NHS GP deployment)
core_hours:
  start: "08:00"            # Core hours start (local time)
  end: "18:30"              # Core hours end (local time)
```

```yaml
enhanced_access_windows:
  - weekdays_evening
  - saturday
red_flag_set:
  - "chest pain"
  - "shortness of breath"
  - "severe bleeding"
  - "unresponsive"
  - "suicidal ideation"
  # (add all high-risk symptoms/phrases requiring 999/A&E diversion)
priority_thresholds:
  stat: 0.9
  urgent: 0.7
  soon: 0.4
  routine: 0.0
sla_targets:
  stat_immediate: "PT0M"              # STAT: immediate action (no delay)
  urgent_first_contact: "PT2H"        # URGENT: within 2 hours
  soon_same_day: "PT12H"              # SOON: within 12 hours (same working
day)
  routine_initial_response: "P2D"     # ROUTINE: within 2 days
hold_back_fraction: 0.10              # 10% slots initially held for urgent/
micro-release
fairness_floors:
  telephone_min_fraction: 0.15        # At least 15% of appointments for
telephone requests
  interpreter_support: true          # Practice has interpreter/language
support available
privacy_policy:
  retention_days: 3650               # Retain audit data ~10 years
  minimize_data: true
# Only store necessary data (transcripts not saved if not needed, etc.)
ooh_policy:
  accept_submissions: true          # Allow patients to submit requests OOH
(deferred triage)
  patient_message: >                # Message shown OOH
    "Outside core hours. For urgent medical help, contact NHS 111 (24/7) or
call 999 for emergencies.
     You may submit your issue now and we will review it when we re-open."
callback_windows_by_priority:
  stat: "immediate"
  urgent: "within_2h"
  soon: "same_day"
  routine: "within_48h"
accessibility_and_language:
  interpreter_languages: ["en", "ur", "pa", "pl", "ar"]  # Languages
available for interpretation (English, Urdu, Punjabi, Polish, Arabic, etc.)
  offer_bsl: true                                      # British Sign
Language support offered
  collect_patient_prefs: ["female_clinician", "wheelchair_access",
"language_preference"]
```

```yaml
  # These patient preferences will be asked/recorded and considered in
matching.
safety_gate:
  model_ner: "bioclinicalbert"
# Transformer model for symptom/entity extraction (multilingual)
  model_classifier: "medalpaca-sm"     # Transformer model for emergency
classification
  multilingual_mode: "native"         # Use native multilingual models
  translate_engine: "none"            # Not used (only if multilingual_mode =
translate)
  red_flag_threshold: 0.65            # Confidence threshold for red-flag
entity detection
  emergency_confidence: 0.70          # Confidence threshold for emergency
classifier
  acuity_model: "xgboost"             # Ensemble acuity prediction model
  acuity_threshold_emergency: 0.75    # Threshold for acuity model tagging
"Emergency"
  timeout_ms: 800                     # 0.8s timeout for safety gate
processing
  fallback: "rules"                   # Use rule-based fallback if ML times
out or fails
triage:
  score_weights:                      # Weights w1..w5 for triage composite
score
    acuity: 1.0
    risk: 0.5
    complexity: 0.2
    time: 0.5
    capacity: 0.2
  aging_interval: "PT5M"              # Recompute triage scores every 5
minutes for SLA aging
  sim_threshold: 0.8
# Similarity threshold for deduplication (80%)
  dedup_window: "PT72H"               # Look back 72h for duplicate requests
provider_assignment:
  weights:                            # Weights α..ζ for provider selection
scoring
    availability: 1.0
    workload: 1.0
    continuity: 0.5
    resolution_rate: 0.2
    distance: 0.2
    fairness: 0.5
telephony:
  ivr_intent_classifier: "default"    # (Could specify model or logic for IVR
intent classification)
  max_callback_retries: 3             # Try calling back up to 3 times before
escalating
  emergency_transfer_enabled: true    # IVR can transfer STAT cases to
emergency line
ambient_scribe:
```

```yaml
  enabled: true
  asr_model: "whisper-large-v3-medical"
  asr_diarization: true
  llm_model: "medpalm2"
  max_summary_tokens: 2048
  uncertainty_highlight: true
  require_clinician_approval: true
  store_audio: "binary"
  terminology:
    condition: "SNOMED-CT"
    medication: "RxNorm"
  timeout_ms: 30000
  fallback_mode: "transcript"
security:
  audit_all_events: true            # Audit every important event (ingress, decisions, etc.)
  worm_audit_store: true            # Use write-once immutable audit storage
  auth_required: true               # All API calls require auth (implied)
  encryption: "TLS1.2+"             # Data encryption standard (in transit and at rest)
observability:
  metrics_collection: true
  slo:
    triage_p95_ms: 2000             # Triage service 95th percentile latency target 2s
    scribe_draft_p95_s: 60          # Scribe draft target 60s
    portal_core_uptime: 0.999       # 99.9% core hours uptime target
    audit_coverage: 1.0             # 100% of events audited target
  alerts:
    triage_latency_breach:          # Alert if triage p95 >2s for 5+ minutes
      threshold_ms: 2000
      duration: "PT5M"
    scribe_backlog:                 # Alert if too many pending scribe tasks
      threshold_queue_length: 5
      duration: "PT1M"
    portal_downtime:               # Alert if portal down in core hours > X minutes
      threshold_minutes: 5
    fairness_monitor:
      telephone_fraction_floor: 0.15  # If phone fraction below 15% over a period, flag it
      check_window: "P1D"             # Check daily
```

(Explanatory comments in-line as shown would typically be omitted or transformed in an actual config file, but are included here for completeness. In production, documentation would accompany the config.)

This YAML configuration encapsulates the entire setup. It can be loaded by the system on startup or per deployment. Each key corresponds to a module of functionality: - `core_hours`, `ooh_policy` for front-door access rules. - `safety_gate` block containing all SafeguardGate tuning. - `triage` block with scoring weights and dedup. - `provider_assignment` for fairness and continuity weights. -

`telephony` for IVR/callback specifics. - `ambient_scribe` for the scribe feature. - `security` and `observability` for overarching assurance and monitoring.

It uses the same keys as the design's initial snippet (e.g., `hold_back_fraction`, `fairness_floors`) and introduces nested structures for clarity (grouping triage and assignment related configs). All default values given reflect the rationale discussed.

Clinical sign-off items: These would be annotated or highlighted in documentation. In this config, those requiring sign-off include: `red_flag_set/threshold`, `emergency_confidence`, `acuity_threshold_emergency`, `priority_thresholds`, `sla_targets`, `ooh_policy.accept_submissions` (policy decision), all `safety_gate` thresholds, `triage.score_weights` (impacting prioritization), `provider_assignment.weights` (affecting care distribution), and `ambient_scribe.require_clinician_approval` (must remain true). In practice, we would tag these in the config file or maintain a separate safety checklist mapping to each key.

5. Open Issues and Assumptions

Despite our comprehensive defaults, a few open issues and assumptions need to be acknowledged. These will be addressed through further testing, calibration, or policy decisions:

- **Calibration of Scoring Weights:** The exact values for `triage.score_weights` (w1–w5) and `provider_assignment.weights` (α–ζ) are assumptions based on expert judgment. **They require retrospective calibration** using historical triage outcomes and provider assignment data. For instance, we will run the system in **shadow mode** alongside current practice to adjust these weights so that the AI's priority assignments match clinicians' judgments in a majority of cases. Until such calibration is done, the defaults should be treated as educated starting points (not final). This is an open issue: a shadow trial or at least a retrospective test on past cases is needed to finetune these parameters. We assume linear weighting is sufficient; if not, more complex models (like nonlinear scoring or additional features) might be introduced.

- **Acuity Model Performance:** We assume the acuity model (XGBoost) is well-trained and calibrated to output a meaningful probability of emergency/urgency. If local data or patient populations differ (e.g. the model was trained on hospital triage data, which might not directly apply to GP context), its threshold (0.75) might need adjustment. Open question: do we need to retrain or fine-tune the acuity model on GP triage data? Likely yes in the long-term. For initial deployment, we will run it in shadow and measure its predictions. We've assumed it generalizes; this needs confirmation. We plan to use a **retrospective dataset of GP consultations labeled with urgency** to validate its specificity/sensitivity and recalibrate thresholds (Platt scaling or similar) if needed [3].

- **Emergency Classifier Scope:** The `MODEL_CLASSIFIER` "medalpaca-sm" – it's assumed to detect obvious emergencies from text. We should verify its coverage of scenarios. It likely doesn't cover non-textual cues (like if patient sounds very ill on phone – which ASR text might not capture fully). We assume textual description suffices, but that's an assumption. In practice, we rely also on the patient's self-selection (if they call 999 themselves) – but since our system explicitly tries to catch what the patient might not realize, we need to monitor for any **false negatives** (cases that were emergencies but neither lexicon nor classifier triggered). All such cases (hopefully zero) in pilot should be analyzed to see if threshold or lexicon update could have caught them. So an open action: continuously update `red_flag_set` with new terms or phrasing patterns that emerge (especially for free-text inputs).

- **Symptom Lexicon Maintenance:** The `red_flag_set` list is provided as a static config, but it will require ongoing updates as clinical guidelines evolve or new terms become relevant. For example, if NICE flags a new COVID symptom as a red flag for urgent assessment, we need to add it. We assume the initial list covers major emergencies (chest pain, etc.). It should be reviewed by clinicians regularly (e.g. every 6 months or when new safety alerts come out). We should maintain versioning for this lexicon, possibly even link it to NHS 111's keywords. This is an assumption that the list is exhaustive enough to start with (open issue to validate with clinical safety officer and maybe cross-reference NHS Pathways keywords).

- **Fairness Floor Values:** The 15% telephone floor is somewhat arbitrary. It assumes perhaps 15% of patients are phone-first. Some practices might have 30% elder patients who phone; others might have 5%. Thus, **local policy** will decide this number. We have an open issue to integrate practice demographics and desired equity targets when setting `telephone_min_fraction`. We recommend initially to observe metrics: if phone demand is higher than floor, floor doesn't kick in (no issue). If phone demand is consistently lower, maybe our digital inclusion efforts are succeeding and we could lower floor – but careful: the floor is a minimum, it doesn't harm if it's lower than actual demand. If phone demand is higher than supply, the floor might need raising or alternate strategies. In short, the floor should be calibrated or at least confirmed with practice managers. Similarly, other fairness measures (like ensuring certain cohorts – e.g. care home patients – get a minimum service level) might be added once data is gathered. We assume the simple 15% phone and language support cover main fairness for now, but acknowledge likely further tuning.

- **Capacity Shaper Threshold (Significant delta ε):** In the capacity shaping pseudocode [83], there's a function `Significant(delta, eps)`. We have not explicitly set `eps` (epsilon). This threshold determines when an imbalance triggers a micro-release of slots. For now, we assume the implementation will consider a delta (difference between needed slots and available slots) "significant" if, say, > 1 slot or >X% deviation. We should define this in config (perhaps implicitly or as a constant). It's an open technical detail: the system likely uses something like `if abs(delta) > 1.0 (in some normalized units)`. We will clarify that in implementation. In any case, this doesn't require clinical sign-off, but it's a tuning parameter that operations staff may adjust. We will default it to ensure *any urgent shortfall triggers releases immediately* (so likely eps = 0 for urgent category, and maybe eps = small fraction of routine capacity for routine imbalance).

- **OOH Submission Policy:** Accepting submissions out-of-hours is set to true by default (with warnings). However, this is a policy decision that some regions might overturn. **Assumption:** The practice has arrangements to safely handle deferred requests first thing in the morning. If a practice is not confident about that, they might turn this off, forcing patients to use NHS 111 at night. We built our system to allow OOH deferrals because NHS England's direction is 24/7 digital access, but it's an open point whether local governance will allow it day one. We mark it clearly for sign-off. If it stays true, we'll run in shadow initially to ensure none of those deferred queries turn out to be emergencies that slip through. (Perhaps double safety: the OOH message strongly advises what to do for urgent needs – we assume patients heed that, which is a risk assumption relying on patient behavior). We might implement a morning report highlighting any OOH submissions and their content, with an automated re-run of SafeguardGate at 8am to double-check none of them were actually urgent (this could be a further mitigation if `accept_submissions:true`). This is an idea to consider.

- **LLM Hallucination and Data Protection:** While we have mitigations (uncertainty highlights, human approval), it's worth noting that LLMs can sometimes output text that wasn't said (hallucinations). We assume Med-PaLM2 or similar with our prompting (like "cite from transcript" style) keeps this to a minimum [56]. But open issue: we need to validate the **fidelity** of summaries. In pilot, we should measure how often clinicians need to edit the AI draft and why. If hallucination or misrepresentation is frequent, we may need to adjust the prompt or switch to a more constrained summarization (like fill-in-the-blank template). Our fallback mode "template" is available if needed. So a contingency: if the LLM proves unreliable in early use, we might temporarily set `fallback_mode: "template"` or even disable the summary, until fixed. We assume given the state of 2025 LLMs and the fact that clinicians review everything, the risk is manageable, but we remain cautious.

- **Model Updates and Drift:** We assume model performance will remain consistent, but in reality, any ML model can drift as medical language/slang evolves or as the data distribution changes. We have monitoring in place (e.g. log cases where SafeguardGate might have missed something or where clinicians disagree with AI output). An open point is establishing a **model recalibration schedule**. For example, re-evaluate the emergency classifier and acuity model every 6-12 months on new data, and update thresholds or retrain as needed. We should also watch for bias in these models – e.g. does the emergency classifier work equally well for non-English transcripts if using native mode? The design mentions tracking per-language performance [7] – so we will gather that data. If say performance in Urdu transcripts is lower, we might switch to `translate` mode for those (or fine-tune a model). So there's an implicit assumption that `MULTILINGUAL_MODE: native` is best; open to change if evaluation shows otherwise.

- **Integration with existing GP workflows:** We assume the creation of a FHIR Task and assignment to a clinician's queue (as per design) is sufficient to integrate with their workflow [88]. If clinicians use a different task management system or an EHR that doesn't show FHIR tasks in real time, we might have to adapt (e.g. send an email or use an existing worklist). The config doesn't cover this, but it's an operational assumption. We note it because if not integrated well, delays could occur. This is not a parameter issue, but an implementation detail to verify.

- **Patient Communication & Consent:** We assume patients will understand the messages given (like the OOH message, callback window explanations, etc.). This is an open area where we might need to adjust wording based on feedback (health literacy). E.g., does "initial response within 48h" cause confusion? Should it be "within 2 working days"? This might be refined in the `callback_windows_by_priority` text or patient FAQs. Also, consent: we assume that asking consent for scribe at the start of consult suffices (which is inline with NHS guidance) [66]. If some patients prefer not, the clinician can disable scribe for that encounter. We will log consent outcomes to see if uptake is an issue.

- **Shadow Mode Requirements:** We flagged that some components (triage algorithm, scribe) essentially should run in shadow mode or at least in a controlled rollout ("canary") before fully live. We assume we'll have that opportunity (which is a plan item, not a given – but from a safety perspective, we insist on it). Open item: define **shadow mode success criteria** – e.g. AI triage agrees with nurse triage on priority in, say, ≥95% cases with no critical misses, for 2 weeks, before turning on automated routing. Similarly for scribe: maybe start with a subset of clinicians and collect feedback. Our config supports enabling/disabling features easily (e.g. `SCRIBE.ENABLED`). We note that while defaults are set to on, we can and likely will turn `SCRIBE.ENABLED=false` during initial go-live and enable it gradually after calibration. This phased approach is assumed in our plan.

- **Local Policy Variables:** Some values will explicitly depend on local decisions:

- Use of interpreter and languages in `accessibility_and_language` should reflect the local population served. We put some common ones, but for example if a practice is in an area with many Bengali speakers but no Polish patients, they'd adjust the list. It's assumed they will do so.
- `collect_patient_prefs` keys: we included female clinician preference, wheelchair access, etc., which are common. If a practice or PCN wants to include other preferences (e.g. "prefers own GP vs any GP"), they can extend that. Our system can handle it as long as it's coded in provider data as well. We assume these cover the main things that affect assignment fairness.

- The decision whether to allow direct 999 transfer (emergency_transfer_enabled): some sites might not want the IVR to automatically call an ambulance for liability reasons. We default true because if we detect a STAT, time is of the essence. But that's a policy call. Our assumption is that integration with emergency services exists and is allowed under proper governance.

- **Continuity vs Speed trade-off:** We assume moderate weight on continuity is desired (we set 0.5). However, GP practices vary: some might insist patient always gets their own GP for continuity except in emergencies. Others might not care who sees the patient as long as it's quick. Our config can accommodate either by adjusting `provider_assignment.weights.continuity`. It's open which approach each practice will prefer. Our default is a middle ground. We will confirm with the practice leadership; this might require retrospective analysis of outcomes (does continuity weight of 0.5 yield significantly more follow-ups resolved? Or if continuity is over-weighted, does it slow down response for some patients?). We assume the provided default is safe and somewhat balanced, but adjustment is expected once there's some data (that's fine since it doesn't compromise safety, just experience).

- **SIM_THRESHOLD for Dedup:** We set 0.8 arbitrarily. This is an area for fine-tuning. If it's too high, some duplicates might slip through (two very similar requests might be considered different if similarity only 0.75). If too low, distinct issues might accidentally merge (which could be a minor safety issue – but we do keep both info in merged task, so maybe not too bad). We assume 0.8 is reasonable for narrative text; we'll test on sample e-consults (for example, if a patient submits "cough for 3 days" twice, that should definitely merge with >0.9 similarity; if they submit "cough" one day and "fever" next, that likely will be low similarity and shouldn't merge). We'll adjust tau if needed. This is an open technical tuning issue, with no critical safety impact as long as it's relatively high (false merges are more concerning than false non-merges).

- **Fallback Plan Activation:** We assume fallback mechanisms (like SafeguardGate fallback=rules, scribe fallback=transcript) will handle worst-case scenarios gracefully. But we should test those modes explicitly (simulate model failure or high load). Also, any "none" option (like if we set fallback to none, or if clinician rejects scribe and we then auto-approve transcript) should be validated. We have an assumption in scribe pseudocode that if clinician rejects, we either give them the transcript or a blank note and then mark as "approve" (meaning they will fill manually) [89]. We consider that a final fallback. This seems sound. Open check: ensure that in fallback or partial failure modes, audit events are still recorded so that we can review them (e.g. how often did rules fallback trigger in SafeguardGate? Ideally rarely, but we need to track it).

- **Clinical Safety Case Outstanding Items:** We will need to formally document some of these decisions in the DCB0129 hazard log. For example, hazard: "AI triage assigns incorrect lower priority → delay in care". Mitigation: thresholds calibrated, human override possible, SLA aging ensures eventual escalation, etc. We assume these mitigations suffice, but during the safety case review, additional controls might be recommended (e.g. maybe initially require clinician to review AI triage for stat/urgent tasks as a double-check – effectively a shadow confirmation; or perhaps run periodic audits of outcomes). We have to remain flexible to incorporate any such recommendation. That's not a parameter per se, but a process – e.g. we might have a config `shadow_triage_mode: true` which means even though AI sets priority, a clinician quickly reviews high priority ones. Or use `HOLD_BACK_FRACTION` in another way (like to hold urgent cases until clinician verifies, but that would defeat the purpose of speed; likely not, we'll instead monitor).

In conclusion, none of these open issues indicate a flaw in the design; they are points for **verification, local tailoring, and continuous improvement**. We've built in configuration hooks for all of them, meaning we can adjust values without code changes as we learn more. Key values requiring **retrospective calibration** or **local policy input** have been identified: triage and assignment weights, similarity threshold, fairness percentage, continuity emphasis, and OOH submission policy. We assume initial "conservative" values and will then iterate.

We also assume the system will operate in a **"soft launch" or shadow mode** for a period, which is absolutely critical for patient safety – this assumption should be turned into a concrete plan (as described next). During that phase, we'll treat these open points as experiments: measure, adjust, and finalize the numbers with confidence.

6. Validation and Calibration Plan

To ensure the configured system is safe, fair, and effective, we propose a comprehensive validation and calibration plan. This plan covers offline testing, shadow mode validation, ongoing monitoring, and established clinical approval workflows:

6.1 Pre-Deployment Validation (Simulation & Offline Tests)

- **Retrospective Simulation:** We will collect a set of historical patient requests (e.g. e-consultation data or call logs) with known outcomes or clinician-assigned priorities. Without deploying live, we run these cases through the AI triage pipeline using our default config. We'll compare:
- AI-assigned priority vs actual priority/outcome.
- Cases where AI would have diverted to 999 vs what happened (should catch any actual emergencies).
- Distribution of priorities to see if it matches expectations (e.g. not everything ends up urgent).

This simulation allows us to adjust `PRIORITY_THRESHOLDS` or weights if, for example, we find AI scoring too high or low. It also validates SafeguardGate: any known emergency case from the past should be diverted by our system. If any aren't, that's a red flag to tweak thresholds or lexicon.

- **Edge Case Testing:** Construct synthetic edge scenarios to test safety:
- A text input containing multiple red flags (e.g. "severe chest pain, difficulty breathing, high fever") – ensure SafeguardGate triggers diversion (with either lexicon or classifier).
- A borderline score scenario (score exactly at 0.69, just below urgent threshold) – ensure it maps to soon and that after aging say 1 hour it crosses 0.7 to urgent. This tests monotonic escalation.
- Capacity extremes: simulate if no provider available vs many providers – ensure no bizarre priority inversions (the priority label should remain primarily driven by clinical factors, capacity just influences slowly via scheduling).
- Multiple duplicate submissions: ensure dedup merges them and audit logs it [90].
- Ambient scribe error paths: feed a very long conversation or poor audio – does it time out and properly fallback to transcript? Induce an ASR or LLM failure and verify `fallback_mode`

engages (transcript is delivered for approval). We will simulate that by perhaps disabling the model temporarily and seeing system behavior.

These test cases will be documented. If any fail, we refine config or code logic.

- **Performance Testing:** Using dummy data, ramp up volume (e.g. 100 simultaneous triage requests) to see if p95 latency meets 2s target [78]. If not, either increase resources or adjust TIMEOUT_MS if needed. This ensures that under expected peak load (say Monday 8am surge), the system still performs. We should also test a large number of concurrent scribe tasks to ensure the 60s draft target holds (if not, we may need to queue or reduce concurrency, which is more an infra scaling issue but config like `SCRIBE.timeout_ms` might be adjusted if infrastructure can't support it – though we have it at 30s, giving slack to 60s SLO).

- **Security Testing:** Verify audit logs are written for all events:

- Create a test submission → ensure an `audit.success` event was logged with proper details [75] [91].
- Try an unauthorized action → ensure `audit.access_denied` is logged [77].
- Check that consent denial triggers audit and stops processing [81].

- Run through an entire scenario and trace correlation IDs to confirm end-to-end traceability (maybe using the audit ledger). This validation ensures our `security.audit_all_events=true` and related settings indeed produce 100% coverage [78].

- **Clinical Safety Review (Pre-deployment):** Convene a meeting with the Clinical Safety Officer (CSO) and relevant GPs to walk through a sample of triage outputs and scribe outputs. This acts as a qualitative validation:

- Show them how the SafeguardGate would respond to certain phrases – get their agreement that the diversion threshold is appropriate. If clinicians say "0.65 might be too low, we'd get too many false alarms" or conversely "I'd rather it be even more sensitive", we incorporate that feedback (with evidence if possible).
- Present the priority mapping on a set of example cases (perhaps mild, moderate, severe symptoms) – ensure they agree with categories (this basically validates `PRIORITY_THRESHOLDS` in clinical terms).
- Show a demo of ambient scribe: let them see a transcript and the generated summary, and ensure they are comfortable with the level of detail and that mandatory approval step. Also confirm consent process is acceptable (maybe they want an on-screen prompt "Patient X has consented to recording – yes/no" which we have).

This session might reveal needs for small config tweaks (e.g. adding a symptom to red_flag_set, adjusting message wording, etc.). We document those and adjust before live.

6.2 Shadow Mode Deployment

- **Initial Shadow Phase:** When we first deploy to a live environment, we will run the AI in parallel to human processes:

- **AI Triage Shadow:** Patients continue to be triaged by clinicians (or existing process), but simultaneously their data is run through the AI triage. The AI's priority recommendation is logged but not acted upon automatically. We then compare each case's AI priority vs actual outcome. Any discrepancies, especially where AI would have given a **lower** priority than human (potential under-triage), are flagged and reviewed. Over, say, a few weeks or a certain number of cases (e.g. 200 cases), we'll gather statistics. We expect some differences – if systematically AI is lower for certain scenarios, we might lower a threshold or increase a weight. If AI is mostly matching or higher (over-triaging a bit), that's safer though not efficient – we could consider tuning down if it's extreme, but initial tolerance for some over-triage is fine.

  - We will produce a confusion matrix of AI vs human priority for this period.
  - Specifically, check that **no urgent case was classified routine by AI**. If any found, that's a serious issue to fix before go-live (maybe needed lexicon or threshold tweak).
  - If AI flagged something as urgent but human triager did not, those cases should also be reviewed – was AI correct (then the human possibly missed something, or AI is oversensitive)? If AI was correct (the patient ended up coming back worse or had to be escalated later), that's evidence to keep AI thresholds as is or even lower them for more sensitivity. If AI was simply too sensitive (false alarm but safe), we note it but likely keep config (we prefer false alarm over miss).
  - These cases are used to refine the RiskClassifier or lexicon if needed (e.g. AI thought "dizziness" is a red flag, but clinicians say it isn't by itself; we might adjust RiskClassifier or remove that word from lexicon if it was in).

- **Scribe Shadow:** In the first phase, clinicians will use the ambient scribe but not rely on it. For example, they conduct the consult with recording on (with consent), the AI produces a draft, but clinicians still write their own note and then compare to the AI draft. Or the clinician edits the AI draft heavily. This is essentially a trial to see how accurate and useful it is. We will:

  - Collect feedback from each clinician: Did the AI miss anything important? Did it hallucinate anything incorrect? How much editing was needed (minor vs major)? We might even have a feedback form integrated ("Was the final note mostly AI-generated or mostly rewritten?").
  - Monitor time saved: Did having the draft reduce documentation time or was fixing it more trouble? This qualitative feedback will determine whether to fully deploy. The threshold for moving from shadow to active usage might be, for example, if >80% of notes can be used with only minor edits and no safety issues identified.
  - Also verify technical aspects: ensure audio quality is adequate, WER is as expected (we can have someone manually verify a subset of transcripts vs audio to compute actual WER).
  - If any safety concern arises (e.g. AI consistently mis-summarizes medication doses), we *pause* and fix that (maybe update the LLM prompt to be more literal, or adjust extraction logic).
  - Only once clinicians are comfortable (which is a key part of clinical approval – they must sign-off that the scribe doesn't introduce unacceptable risk) would we consider using it as a default documentation tool. Even then, we maintain **mandatory approval** always.

- **Iteration:** During shadow mode, we treat the configuration as in flux:

- Adjust thresholds or weights and rerun some cases to see if alignment improved.
- Possibly A/B test small changes: e.g., try `emergency_confidence 0.6` for a week and see if more diversions occur, were those justified or false alarms? The audit logs of `urgent_diversion` events [80] can be checked against clinician outcomes or patient follow-up.

- Similarly, fairness: monitor if telephone patients' waiting times are in line with online patients. If not, maybe raise the telephone_min_fraction or check if scheduling is actually honoring it (the co-pilot might need to enforce it by reassigning slots midday).

- **Clinical Sign-off (Go-Live):** After a defined evaluation period, we compile the results (e.g. "AI vs nurse triage agreement 97%, with no critical discrepancies; ambient scribe drafts accepted with minor edits 90% of time, major edits 10% but caught by review, etc.") and present to the Clinical Safety Officer and practice clinical lead. They will review the hazard log updated with this real-world data. If all is acceptable, they formally approve moving to live mode:

- At that point, AI Triage can start automatically assigning priority and creating tasks, with clinicians handling them as they come (instead of clinicians triaging).
- The SafeguardGate will actively divert emergencies (and we ensure there's a process that when an IVR says "go to A&E now", an alert is also sent to the practice maybe so they know a patient was diverted – design says it creates a safety alert for staff [80]).
- Ambient scribe could be offered to all clinicians as a tool (some might still opt out, that's fine – it's optional per encounter via consent).

The sign-off at this stage might be conditional – e.g. "okay to go live, but please monitor X metric closely and report weekly to us for first month".

6.3 Post-Deployment Monitoring (Continuous Validation)

Once live, we will continuously monitor and periodically calibrate:

- **Safety KPIs Monitoring:** Using the observability config, we will track:
- Number of SafeguardGate diversions per week, and audit if they were appropriate (follow up outcome). If any turn out to be false diversion (patient could have been managed in GP), note that (may adjust threshold slightly upward if too many false alarms cause patient anxiety). Conversely, ensure no emergency slipped through (we expect none, but if one occurs, immediate investigation).
- SLA compliance: Are urgent callbacks truly happening within 2h? If not, why – capacity issue or triage mis-labeling? If the latter, recalibrate. If capacity, maybe hold_back_fraction was too low or fairness issue.
- Audit logs: we expect 100% events audited. Our observability should alert if say any event path didn't generate an audit (shouldn't happen by design, but we verify).
- Bias/Fairness metrics: For example, measure median time to response for online vs telephone requests. Our fairness aim is parity, so if data shows phone requests taking significantly longer, we need to intervene (maybe increase that fairness boost in triage scoring or allocate more staff to phone queue). Also watch by demographics if possible (e.g. are non-English requests being handled timely? The system should incorporate interpreter logic but if e.g. translation or availability causes delays, we consider adjustments).

- Model drift signals: e.g. track SafeguardGate false negative rate (very hard since we ideally have none, but any missed emergency = immediate action). Also track the proportion of tasks getting escalated by SLA aging – if a lot of tasks routinely age from routine to urgent, maybe initial scoring is too optimistic; we might lower thresholds or increase weights to push them higher initially. Ideally, aging should catch only outliers, not a big bulk.

- **Clinician Feedback Loop:** Establish a channel (perhaps a quick form or periodic meeting) for clinicians to report any concerns:

- E.g. “Triage labeled this as routine but I felt it was urgent because X” – we gather such cases and feed them into improving the rules or models. Maybe our ML missed a subtlety; we can retrain it later with those as positive examples of urgent.
- Or “Ambient scribe made up a detail about social history that wasn’t said” – immediately examine that and adjust prompt/LLM config to prevent it. Also that specific content would have been edited out by clinician (since approval needed, no harm reached patient record, but we treat it seriously to avoid fatigue).

- Perhaps include a disagreement button in the UI: if a clinician changes the AI triage priority, the system logs that feedback (this can feed into model refinement). Similarly, if a clinician edits a scribe note significantly, we could have a classification of why (structured vs unstructured but this might be too advanced; initially manual feedback is fine).

- **Periodic Recalibration Cycles:** After going live, we plan routine reviews at 1 month, 3 months, etc.:

- Re-run the priority thresholds against realized data distribution and outcomes. Possibly adjust thresholds if, say, we find that “urgent” category is overwhelmed or underused. (However, because these thresholds tie to clinical urgency, we’d be cautious changing them absent a clear reason.)
- Fine-tune ML components: maybe retrain the acuity model on data from the practice/PCN to better predict urgency in this setting – that could improve accuracy and reduce reliance on manual threshold tweaks.
- Expand lexicon or update it (with CSO sign-off).
- Update fairness floors if needed (if 15% was too low or if technology adoption changes).
- Evaluate ambient scribe quality improvements if new model versions become available (the config allows switching LLM/ASR easily, but per NHS AI governance, any such change is like a new device – we’d evaluate offline first, as per Appendix F where new model versions would go through testing and canary deployment [92]).

- Security/consent audit: ensure no breaches, all access logged, retention working (maybe simulate a data deletion after retention period if needed).

- **Clinical Governance Oversight:** As part of DCB0160 compliance, we will maintain a **Hazard Log** and **Safety Case** throughout. For each potential hazard (e.g. “missed deteriorating patient” or “AI note incorrect leading to wrong treatment”), we have mitigations (like SafeguardGate, human approval, etc.) that we will verify are effective through above monitoring. We’ll convene the safety review if any incident occurs or even near miss (say a clinician caught that AI triage was off – we analyze and classify it, update config if required). The CSO must sign off on any changes to clinically significant parameters after deployment as well – meaning our process for config change will involve raising a change request, performing risk assessment, then applying (preferably at a scheduled update time).

- **Fairness & Bias Monitoring:** We will specifically look at outcomes for different patient groups:

- Compare average waiting time from request to resolution across different languages (should be roughly equal if interpreter support is working – if not, maybe need more interpreter resources or adjust fairness boost).
- If possible, measure by age or disability if data available, to check no inadvertent bias (the system itself shouldn’t discriminate, but availability of services like home visit vs phone might cause differences).

- The design mention fairness monitoring as part of MLOps [93] – we will incorporate that. For example, if we find the ML acuity model under-triages certain cohorts (maybe it wasn't trained on enough pediatric data, etc.), we address that (maybe add a rule that if age <1 year, always at least urgent, etc., as a patch).

- **Performance & Load:** Use the observability to ensure the system scales. If alerts trigger (like triage latency >2s), investigate and scale infrastructure or optimize. The target is to maintain those SLOs so that system performance never hinders patient care.

- **Audits and Quality Improvement:** After a period (say 6 months), conduct a formal audit:

- E.g. random sample of 20 cases per priority category – were they handled appropriately? Did any routine tasks escalate that perhaps should've been sooner? Did urgent tasks actually need to be urgent or could they have been routine? Use this to adjust.
- Patient outcome audit: follow up any case that was diverted to A&E – was that appropriate (if the hospital found it non-urgent, we might be over-triggering; if they indeed treated it as emergency, great).
- Survey patient satisfaction perhaps for those who used phone vs online (to ensure parity achieved).

- Clinician feedback survey on ambient scribe usage (is it saving time, do they trust it, any suggestions?).

- **Regulatory Compliance Checks:** Ensure we remain compliant with NHS DCB0129/0160:

- Update the safety case whenever we make a config change that affects risk (the safety officer signs off each relevant change).
- The Data Protection Officer reviews that our consent and data handling match the DPIA. E.g. if we decide to store audio long-term, ensure that's covered by our retention and consent. Right now, we said we store audio via DocumentReference with consent – presumably that's fine under direct care and patient consent. But if any secondary use arises (like using transcripts to improve the model), that would need separate consent or anonymization – not directly a config, but a governance matter.

The validation plan effectively treats the system as a **learning health system** component – initial config is based on evidence and best practices, but continuous measurement and improvement are built in. This is critical given AI components: we will not "fire and forget," we will *monitor outcomes and tune* as a standard operating procedure.

To formalize, we will maintain a **Calibration Log** where we record any changes to parameters like thresholds or weights along with justification (e.g. "Oct 2025: urgent threshold lowered to 0.65 from 0.7 because 10% of clinician-marked urgent were scoring 0.68–0.69, wanted to capture those. Approved by Dr. X (CSO)."). This ensures traceability of configuration changes over time, which is itself an important part of clinical governance and audit trail.

7. Patch Suggestions for Algorithm.md

Finally, to implement these configurations and ensure the design document is up-to-date, we propose specific **patches** to the `Algorithm.md` document at relevant sections:

- **At Section 2.2 Urgent-Safety Gate (pseudocode):** Add explicit use of thresholds in the code. For example, modify the pseudocode as follows: ```diff high_risk_symptom_flag := false for s in symptom_list:
- if SymptomLexicon.isRedFlag(s):
- if SymptomLexicon.isRedFlag(s) and nlp_results.confidence(s) >= RED_FLAG_THRESHOLD: high_risk_symptom_flag := true break ...
- if TransformerModel.classify_emergency(doc) == true:
- if TransformerModel.classify_emergency(doc) >= EMERGENCY_CONFIDENCE: high_risk_symptom_flag := true `This makes it clear that the lexicon check can be augmented with a confidence (if NER provides one), and the classifier uses the configured probability threshold [3]. Also, after obtaining` `` `severity_prediction`: ``diff severity_prediction := AcuityModel.predict(features) # "Emergency" | "Urgent" | "Routine"
- if high_risk_symptom_flag || severity_prediction == "Emergency":
- # Use configured acuity threshold for emergency:
- if high_risk_symptom_flag || (AcuityModel.predict_proba(features) >= ACUITY_THRESHOLD_EMERGENCY): ... (diversion) ... ``` `` ` And add a comment that *`ACUITY_THRESHOLD_EMERGENCY` is calibrated for the acuity model to decide "Emergency" category.* This makes the pseudocode align with our config (instead of relying on string output "Emergency", we tie it to the threshold set in config).

- **At Section 3.1 Unified Triage (pseudocode):** After the composite score line [17], insert a note that:

  ```
  # w1..w5 are configurable weights (see config), tuned to ensure score correlates with urgency.
  # Score increases with acuity, risk, complexity, and waiting time, decreases if capacity is limited.
  ```

  And explicitly mention monotonic escalation: ```diff if IsLifeThreatening(riskFlags, acuityScore): ... (override to stat) ... else:

- priorityLabel := MapPriority(score, PRIORITY_THRESHOLDS)

- priorityLabel := MapPriority(score, PRIORITY_THRESHOLDS) # uses config thresholds

  ```
  Also, in the `AgeOpenTasks` pseudocode [21], add:
  ```plaintext
  # RecomputeScore should include TimeDecay(now - t.created_at) which
  ```
  ```

```
grows with waiting time,
# ensuring score' >= original score. Thus Higher(newLabel, t.priority)
check will only upgrade.
```

This documents the monotonic guarantee.

- **Appendix B (Continuity & Fairness Assignment):** In the pseudocode for SelectBestProvider [94], add comments for each term:

```
  best := argmax_c in candidates of (
       α*Availability(c)        # weight for availability (configurable)
     - β*Workload(c)            # weight for current workload
     + γ*Continuity(patient,c)  # continuity preference
     + δ*ResolutionRate(c, requiredSkills)  # past performance
     - ε*Distance(patient,c.site)         # distance penalty
     + ζ*FairnessBoost(patient.needs, c.capabilities, priorityLabel) #
fairness boost for needs-capability match
  )
```

And after that, a note:

```
# α...ζ are configurable weights (see config FAIRNESS_FLOORS/assignment
weights).
# These should be tuned per site to balance equity vs efficiency.
```

Also clarify FairnessBoost in Appendix B narrative: perhaps in the text right below the pseudocode, add: "FairnessBoost could consider patient's language, gender or other needs vs provider attributes; it returns a value (e.g. 0 or 1) that is scaled by ζ [31]. This ensures, for example, a patient requiring an Urdu-speaking clinician gets a slight scoring boost for clinicians who speak Urdu, supporting equity." – basically summarizing what we implemented.

- **Appendix D (Configuration Summary):** Extend the list to include the new parameters: Currently Appendix D [95] lists Hours, Safety, Scribe, Capacity, Equity, Privacy. We should add or modify:

- Under **Safety:** already lists PRIORITY_THRESHOLDS, SLA_TARGETS, SAFETY_GATE. – *that covers most. We might explicitly add RED_FLAG_THRESHOLD, EMERGENCY_CONFIDENCE if not implied by SAFETY_GATE.* (Though SAFETY_GATE. *likely covers it). To be safe: change "SAFETY_GATE."* to "SAFETY_GATE.* (including RED_FLAG_THRESHOLD, EMERGENCY_CONFIDENCE, ACUITY_THRESHOLD_EMERGENCY, etc.)".
- Add a bullet: **Triage & Aging:** e.g. "`TRIAGE.* (weights, aging_interval, deduplication settings)`" – to capture our new triage.score_weights, aging interval, SIM_THRESHOLD.

- Add: **Assignment:** "`provider_assignment.* (selection weights α–ζ)`" – covers continuity/fairness weights. The updated Appendix D might look like:

```
- **Triage & SLA:** `PRIORITY_THRESHOLDS`, `SLA_TARGETS`,
`triage.score_weights`, `aging_interval`, `sim_threshold`
```

```
- **Safety Gate:** `RED_FLAG_SET`, `SAFETY_GATE.*` (incl. thresholds)
- **Scribe:** `SCRIBE.*`
- **Capacity:** `HOLD_BACK_FRACTION`
- **Equity & Access:** `FAIRNESS_FLOORS`,
`CALLBACK_WINDOWS_BY_PRIORITY`, `ACCESSIBILITY_AND_LANGUAGE`
- **Privacy & Audit:** `PRIVACY_POLICY`, security/audit settings
```

(We include ACCESSIBILITY_AND_LANGUAGE which was not explicitly in summary but is part of config.)

- **Sequence diagrams / flows:** In the Telephony Parity sequence [96], currently it shows "Offer CALLBACK_WINDOWS_BY_PRIORITY". That's fine, but maybe add a note in the text that "These callback windows (immediate/2h/same-day/48h) are drawn from the configurable `CALLBACK_WINDOWS_BY_PRIORITY` mapping, which should mirror the practice's SLA targets for consistency." This ensures readers know where those come from.

- **Mermaid diagrams / pseudo:** possibly no need to change, but if we wanted to reflect monotonic aging: The triage sequence diagram shows a loop "Every 5 minutes: Recompute score by age, update Task priority if higher" [97]. That effectively communicates monotonic escalation. We might annotate "(monotonic escalation ensured by design)" as a caption in the doc if clarity needed.

- **OOH Policy in Section 2.1 Portal Uptime Guard:** The pseudocode already reflects accepting submissions if allowed [45]. We might just double-check it aligns with config naming: It calls `if OOH_POLICY.accept_submissions: ... AcceptSubmissionsWithDeferral ... ShowMessage(OOH_POLICY.patient_message)` which is perfect. We just ensure the config example uses same keys (it does). Perhaps add a comment:

```
# OOH submissions are deferred but allowed (controlled by config).
# Patient receives configured deferral message with emergency advice.
```

This emphasizes the safety message.

- **Ambient Scribe in Section 7.1 pseudocode:** The code currently has model names hardcoded (WhisperLargeV3_Medical, MedPaLM2) [98]. We should modify: ```diff

- ASR_model := WhisperLargeV3_Medical # e.g., Whisper large-v3 fine-tuned for clinical domain
- ASR_model := LoadASRModel(SCRIBE.ASR_MODEL) transcript := ASR_model.transcribe(audio, diarize=SCRIBE.ASR_DIARIZATION) ...
- LLM_model := MedPaLM2
- LLM_model := LoadLLM(SCRIBE.LLM_MODEL) # or OpenClinicalGPT/BioGPT as per config draft_note := LLM_model.summarizeToSOAP(... highlight_uncertainty=SCRIBE.UNCERTAINTY_HIGHLIGHT)

```
And similarly utilize SCRIBE.TIMEOUT_MS in the logic (though the
pseudocode doesn't show how to enforce timeout, we might add a comment:
`# ensure processing completes < SCRIBE.TIMEOUT_MS (e.g. 30s) or abort
to fallback`).
```

```
Also incorporate `REQUIRE_CLINICIAN_APPROVAL`: in step 4 and 6 of
pseudocode, it's inherently enforced. We can add at end:
```plaintext
# Note: The draft note is never committed to final record without
clinician approval (per SCRIBE.REQUIRE_CLINICIAN_APPROVAL=true).
```

And in fallback: `diff`

```
    if clinician_action == "reject":
        # 5) Fallback: if LLM draft is declined or unavailable, use transcript or
template-based note
-       if draft_note == null or LLM_model.failed:
-           draft_note := FormatUtils.transcriptToNote(transcript)
-       else:
-           draft_note := TemplateUtils.blankSOAPNote(transcript)
+       if draft_note == null or LLM_model.failed or SCRIBE.FALLBACK_MODE ==
"transcript":
+           draft_note := FormatUtils.transcriptToNote(transcript)
+       else if SCRIBE.FALLBACK_MODE == "template":
+           draft_note := TemplateUtils.blankSOAPNote(transcript)
        docRef.content := draft_note
        ...
```

This uses the config-driven fallback mode. (Pseudo-syntax might vary, but essence is to tie to config.)

- **Appendix G (SLOs & Alerts):** It lists some SLO examples [78]. We should add one for fairness maybe: e.g.

```
- **Telephone parity:** >= 15% of daily requests handled via telephone
channel (monitor daily; alert if <15% consistently).
```

  Or incorporate that into an existing line. Since Appendix G is examples, adding one bullet about fairness floors would reflect our fairness monitoring. The sources mention "access equity considerations" in sources [99], but making it explicit as an SLO might be useful as a patch.

- **Documentation of config keys:** Possibly add an Appendix or table that enumerates all keys and default values. However, our YAML in section 4 serves that purpose. If the design doc is maintained, adding our YAML example (which we have already in 0.5.1 YAML Example, albeit we extended it) might be done: We can update the YAML example in section 0.5.1 to include the new fields: For example, after `callback_windows_by_priority` in the YAML example [100], add the `triage_score_weights`, `provider_selection_weights`, etc. Given the snippet we wrote, the doc maintainers can incorporate as needed.

- Ensure all added keys are explained in doc text: Perhaps add a short section or expand 0.5 Config listing to mention:

- "Additional configurable parameters include triage scoring weights (to balance acuity vs time, etc.), provider assignment weights (balancing continuity vs fairness vs workload), and aging/dedup intervals. These are advanced settings that can be tuned per practice policy." If needed, we could insert a sub-section in the config part or an appendix entry for them.
```

By applying these patches, the design document will accurately reflect the detailed configuration and safeguards we have elaborated. It will help future implementers know where to plug in these default values and how to adjust them, and it will emphasize the critical points (like thresholds usage, monotonic aging, and sign-off requirements) in the context of the algorithms described.

These modifications and additions to the documentation, combined with the configuration and plans outlined above, ensure that the OneCare AI triage system is configured to **maximize patient safety, transparency, and fairness**, while allowing flexibility for continuous improvement. All critical values are now explicitly defined, rationale given (grounded in NHS standards [19] [20] and best practices), and the plan to validate and maintain these settings is in place, fulfilling the requirements of a safe deployment under NHS England's clinical safety regulations [53].

---

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 17 18 19 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 42 43 44 45 46 47 48 49 50 51 52 54 55 56 57 58 59 60 61 62 63 64 66 67 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 85 87 88 89 90 91 92 93 94 95 96 97 98 99 100 Algorithm.md
file://file_00000000d7486243ae6961ac083f366c

16 GP Contract 2025/26: what you need to know | NHS Confederation
https://www.nhsconfed.org/publications/gp-contract-202526-what-you-need-know

20 41 Commonly asked Questions about booking an appointment - Beacon Primary Care
https://www.beaconprimarycare.org.uk/clinics-and-services/appointments-tests-referrals/appointments-with-the-doctors/commonly-asked-questions-about-booking-appointments/

53 68 86 NHS England » Guidance on the use of AI-enabled ambient scribing products in health and care settings
https://www.england.nhs.uk/long-read/guidance-on-the-use-of-ai-enabled-ambient-scribing-products-in-health-and-care-settings/

65 Regulation of AI scribes in clinical practice - The BMJ
https://www-bmj-com.bibliotheek.ehb.be/content/389/bmj.r1248

84 GOSH-led trial of AI-scribe technology shows 'transformative ...
https://www.gosh.nhs.uk/news/researchgosh-led-trial-of-ai-scribe-technology-shows-transformative-benefits-for-patients-and-clinicians-across-london/